Hardware Description Language

Ranim Tom

November 9, 2026

Contents

1 Introduction **11**

1.1 Why HDL . . . 11
1.1.1 Desing methodolgies . . . 12
1.1.2 VHDL . . . 12
1.1.3 FPGA . . . 14
1.1.4 FPGA Configuration . . . 14
1.2 Desing Flow . . . 15
1.2.1 Behavioral Simulation . . . 15
1.2.2 Gate Level Simulation . . . 16
1.2.3 Wrapping up the desing flow . . . 16
1.3 VHDL is not a software . . . 17
1.4 VHDL Desing . . . 18
1.4.1 Entity . . . 18
1.4.2 Architecture . . . 19
1.4.3 Internal Signals . . . 20
1.4.4 Concurrency . . . 21
1.4.5 Process . . . 22
1.5 Some VHDL Examples . . . 23
1.6 Process Rules . . . 25
1.7 More about VHDL Langugage . . . 26
1.7.1 Bit Vectors . . . 26
1.7.2 Bad code: vector mismatch . . . 27
1.7.3 Code comments . . . 27
1.7.4 VHDL syntax exercice . . . 28
1.8 Summary Introduction . . . 29

2 Sequential and Concurrent VHDL **31**

2.1 Introduction . . . 31
2.2 Processes in Depth . . . 32
2.2.1 Processes Interaction . . . 32
2.2.2 Warning: Asynchronous Feedback . . . 33
2.2.3 Process Rules . . . 34
2.2.4 Process Behavior . . . 36
2.3 Data Types . . . 37
2.3.1 Integers . . . 37
2.3.2 Boolean . . . 37
2.3.3 Litteral Assignment . . . 38
2.3.4 Vector Comparison . . . 39

2.3.5 Vector Aggregation and Concatenation . . . 40
2.4 Sequential and Concurrent Statement . . . 41
2.4.1 If-else . . . 41
2.4.2 Case . . . 44
2.4.3 for loop . . . 46
2.5 Concurrency . . . 48
2.5.1 Conditional Signal Assignment . . . 48
2.5.2 Selected Signal Assignment . . . 50
2.6 VHDL Hierarchy . . . 51
2.6.1 Example: Parity System . . . 52
2.7 Test bench . . . 58
2.8 Summary chap 2 . . . 60
2.8.1 Processes (2.2) . . . 60
2.8.2 Data Types (2.3) . . . 60
2.8.3 Selection Control (2.4) . . . 61
2.8.4 Concurrent Statment (2.5) . . . 61
2.8.5 VHDL Hierarchy (2.6) . . . 62

List of Figures

1.1 Different method to describe a MUX circuit . . . 11
1.2 Desing evolution . . . 12
1.3 VHLD different sets of the language . . . 13
1.4 FPGA device . . . 14
1.5 FPGA device . . . 14
1.6 FPGA development process . . . 15
1.7 Behavioural simulation by using waveforms and set of stimulus . . . 15
1.8 Gate level simulation: output produce by actual gates . . . 16
1.9 Desing flow . . . 16
1.10 VHDL concept . . . 17
1.11 VHDL component . . . 18
1.12 VHDL entity . . . 18
1.13 VHDL architecture with its corresponding entity . . . 19
1.14 Internal signal example . . . 20
1.15 and gate implementation using sequential and concurent style . . . 21
1.16 VHDL process . . . 22
1.17 VHDL NOR gate . . . 23
1.18 VHDL NOR gate . . . 24
1.19 Vectors variables in VHDL . . . 26
1.20 Vectors bad code . . . 27
1.21 VHDL comment . . . 27
1.22 VHDL syntax exercice . . . 28

2.1 Process interaction: sequential execution . . . 32
2.2 General view for process interaction . . . 33
2.3 Asynchronous dilemma . . . 33
2.4 Rule 1: Sensitivity list . . . 34
2.5 Rule 2: signal assignment . . . 34
2.6 Rule 3: Multiple assignment . . . 35
2.7 Process Behavior . . . 36
2.8 Process Behavior . . . 36
2.9 Integer data type in VHDL . . . 37
2.10 Boolean data type in VHDL . . . 37
2.11 Signal assignment . . . 38
2.12 Vector comparison . . . 39
2.13 Vector aggregation and concatenation . . . 40
2.14 If else in VHDL . . . 41
2.15 If else circuit . . . 42

2.16 Simple if statment . . . 42
2.17 Simple if statment: preventing latches . . . 43
2.18 Case statement in VHDL . . . 44
2.19 Case statement in VHDL: valid numerical types . . . 44
2.20 if vs case . . . 45
2.21 for loop . . . 46
2.22 Jumping over an iteration . . . 47
2.23 Conditional signal assignment . . . 48
2.24 Conditional signal assignment equivalency with if statement . . . 49
2.25 Selected signal assignment . . . 50
2.26 Selected signal assignment equivalency using case statment . . . 50
2.27 Schematic hierarchy example . . . 51
2.28 Parity Schematic Example . . . 52
2.29 Parity symbol, and its VHDL component . . . 53
2.30 OR symbol, and its VHDL component . . . 53
2.31 Symbols wiring . . . 54
2.32 Top level entity . . . 54
2.33 OR component declaration . . . 55
2.34 Internal signal declaration . . . 55
2.35 Components instantiation . . . 56
2.36 Explicit port mapping . . . 57
2.37 Test bench for the thermostat system . . . 58

List of Tables

Todo list

intro seq and conc 31
if and latche 42
when and if equivalency 49

Chapter 1

Introduction

1.1 Why HDL

Before starting learning `VHDL`, let's see the idea behind `HDL` in general. Consider a mux circuit shown in Figure 1.1.

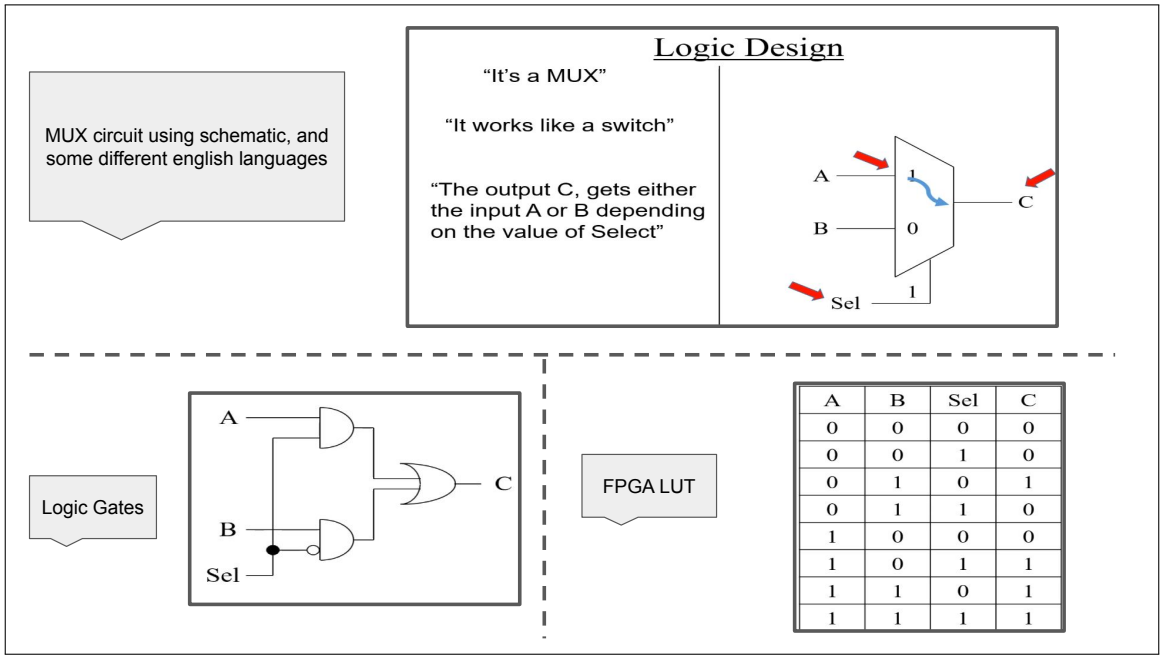


Figure 1.1: Different method to describe a MUX circuit

A MUX can be described using english languages, or the logic gates behind it, or even by LUT. But all these methods are somehow not practical when dealing with some complex design.
The other method which is better to implement our desing is to give some behaviour for the desing (function), and let the tools figure out what logic needs to be used. As long as the tools implement correctly the behaviour, we don't care about the underlying logic. This is the basic idea behind `HDL` such as `VHDL` and `Verilog`.

1.1.1 Desing methodolgies

Throughout time, desing evolves through 3 phases:

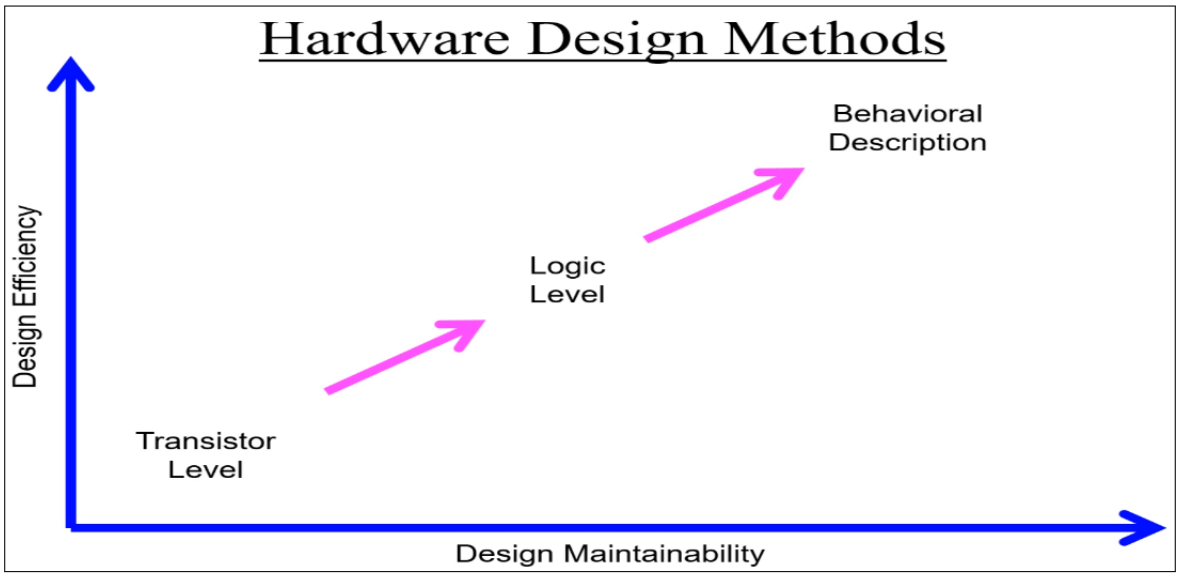


Figure 1.2: Desing evolution

- Connecting transistors manually
- Then use symbol and schematic to simplify the manual connections
  - Such in digital desing courses
- Then move to `HDL` and let the tools figure out the gates and the schematic

1.1.2 VHDL

`VHDL` was created in United states as some defense project for electronic simulation, then later used in logic modeling and simulation.
The most common standard used today is 1993, and recent standard are added after revision in 2006 and 2008.

Simulation and Synthesis:

`VHDL` is a language used for simulation and modeling, and there is allot of features that can't be used in synthesis.

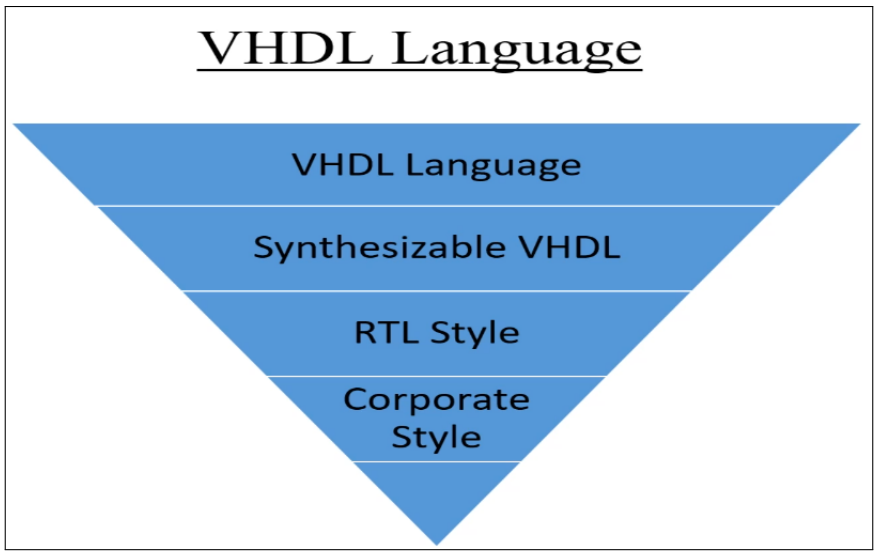


Figure 1.3: VHLD different sets of the language

There is a subset of the language used for synthesis, and we have a specific style used for the logic synthesis called *RTL style*, which will be the focus of this course.
In companies, it is also common to have to whay is called *corporate style*, that is some coding style specific to that company which uses features in certain way. In this course, we focus on Boing style.

1.1.3 FPGA

FPGA is a device composed of many cells, where each cells contains many LUT and FF. The complexity of an FPGA resides in the millions of ways to connect LUT and FF *wihting a cell* and *between cells also*. We use `HDL` and let the tools figures out these connections.

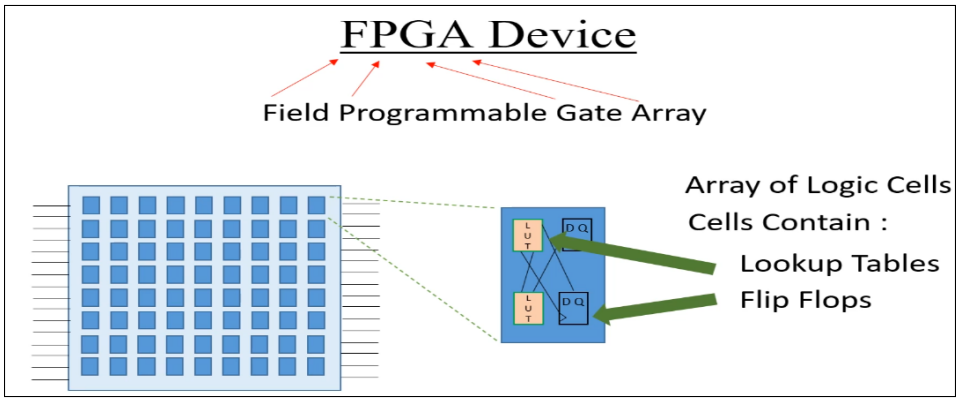


Figure 1.4: FPGA device

1.1.4 FPGA Configuration

By FPGA configuration we mean the way the interconnection is made within the logic cell it self, and from 1 cell to another. This is done by loading these information using some static RAM (or any type of memory), where the SRAM can be transfer using flash memory, or JTAG cable, or some through some processor protocol like `SPI`.
Once these configuration are transfer to make the connections, the connections are made throughout some siwthch boxes, within the cell itself, and from 1 cell to another.

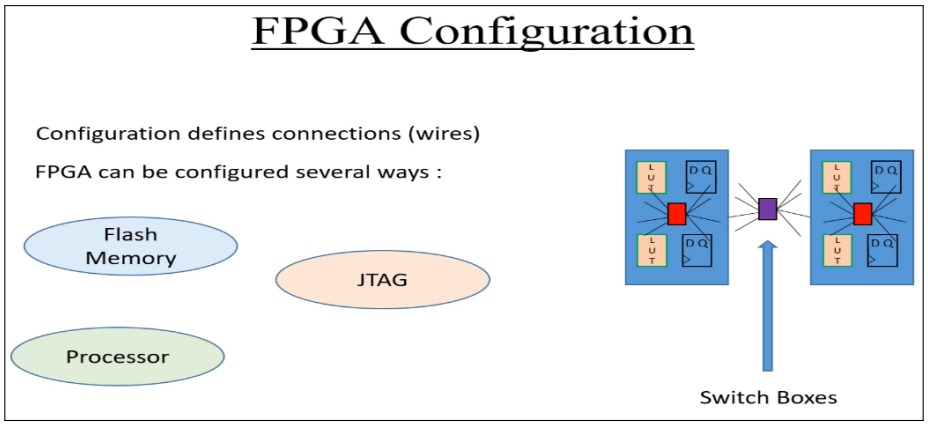


Figure 1.5: FPGA device

This step is defined and implemented is the 3rd step of the FGPA desing flow (1.2), called *place and route*.

1.2 Desing Flow

Now we turn to the FPGA development process. We have 3 phases, as shown in Figure 1.6.

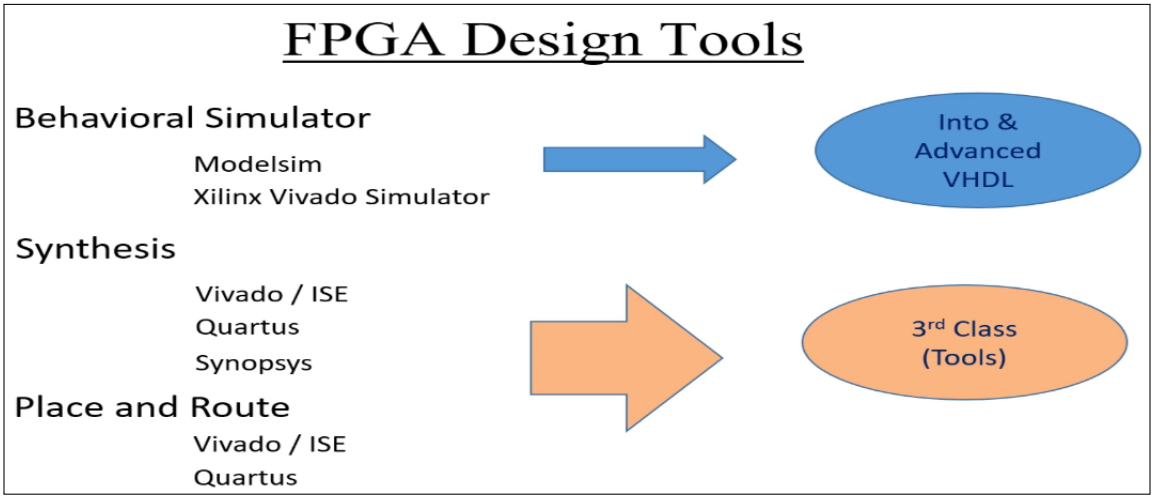


Figure 1.6: FPGA development process

- The 1st step is the *behavioral simulator*, that is we simulate behavioral description of our desing using tools such as `Modelsim` or `Vivado` from `AMD`.
- Once we make sure that the desing simulate the behavior correctly, we translate the behavioral description into logic.

  This is done using *synthesis tools*, such as `Vivado` also.
- 3rd step: place and route

1.2.1 Behavioral Simulation

In the 1st part of the course, we will focus on behaviour simulation.

The idea behind behavioral simulation is that we are simulating the behaviour of the `VHDL` code, and not the logic synthesised yet.
This is done using waveform visualisation, that is giving our code a set of input (stimulus), and then we hope that our code produce the correct result, and be able to visulaize this using the waveforms.

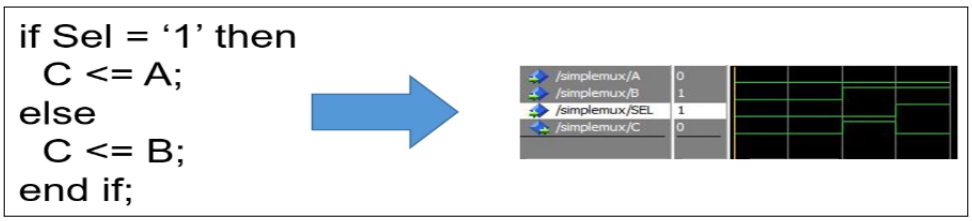


Figure 1.7: Behavioural simulation by using waveforms and set of stimulus

1.2.2 Gate Level Simulation

This is the part we simulate tha actual logic synthesis by the tools. The goal is to hope that gate simulation mathces the behaviour simulations. It is always better to do both, and with experience we gain some confidence that our `HDL` code produces the correct gate simulaitons.

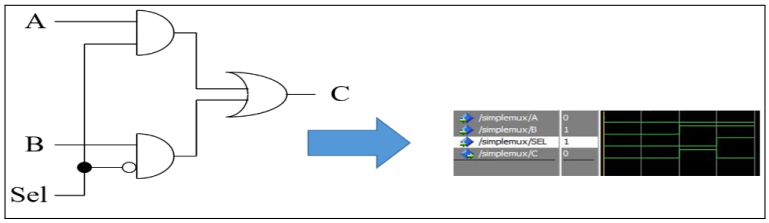


Figure 1.8: Gate level simulation: output produce by actual gates

What can go wrong?

- Sometimes we describe things that can't be translated into logic
- After synthesis, timing problems.
  - We learn how to correct these issues in another part of the course

1.2.3 Wrapping up the desing flow

Figure 1.9 presents another view of the development process seen in Figure 1.6.

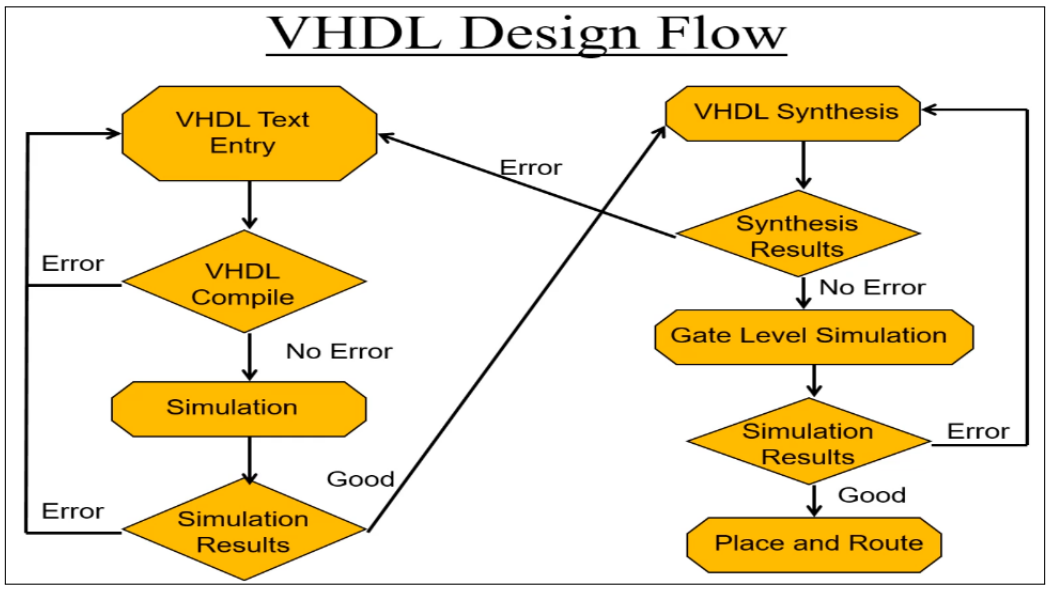


Figure 1.9: Desing flow

- Before the running the simulation part (behavioural simulation as see in 1.2.1), there is the compilation step where we can check for errors like syntax errors, langauges rules,$\cdots$
- Same concept for gate level simulation, we have to pass also the synthesis result step

1.3 VHDL is not a software

It is important to keep in mind is not a traditional programming language, that is the final output is not binary or some executable like a `C` program, which runs on some machine. As a matter of fact, we need to think of `VHDL` as some kind of desing tools which let us *describe the behaviour of our desing*, and then using vendors tools we synthesis this behavour into actual gates and circuit.

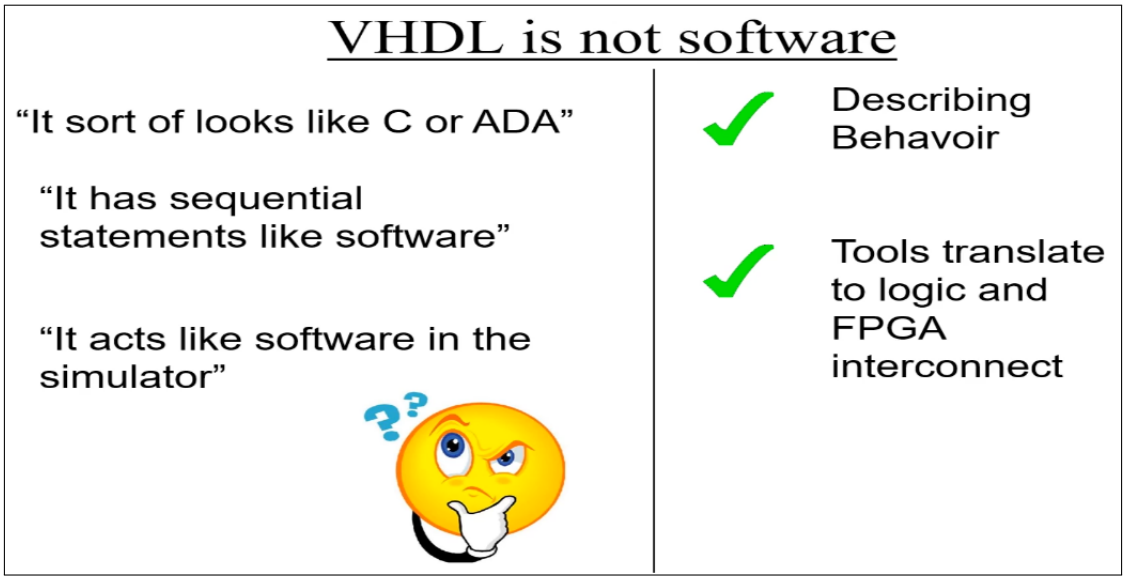


Figure 1.10: VHDL concept

So in other words, `VHDL` is a tools which produces circuit, and not some binary.
After experience, we can imagine what is the circuit behind our `VHDL` code lines .

1.4 VHDL Desing

Now we move more toward the desing itself and VHDL language.

There are 2 blocks to have a correct VHDL module as seen in Figure 1.11:

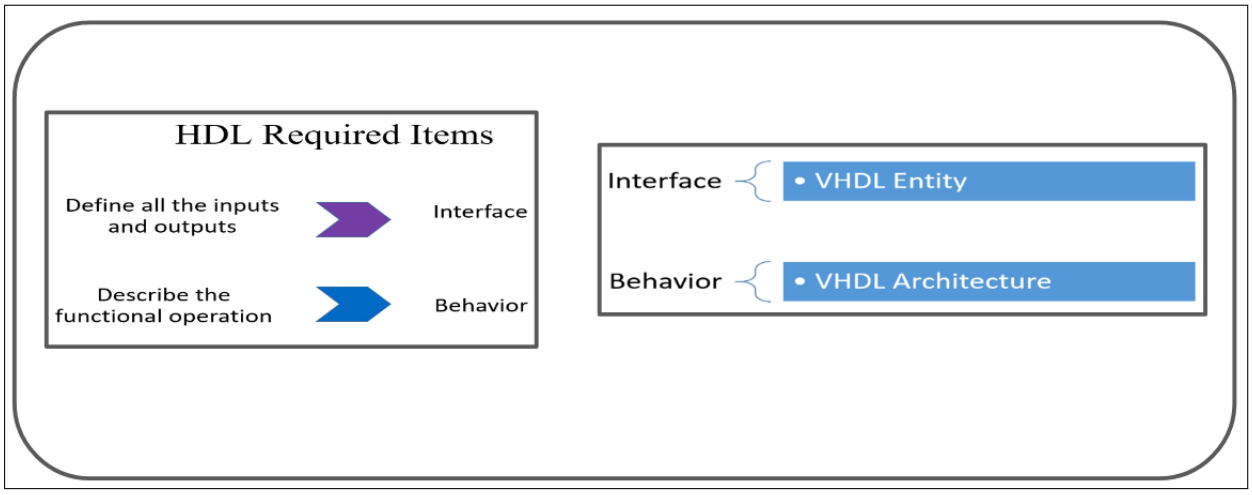


Figure 1.11: VHDL component

- The input and output part, which are structured in *entity* unit
- The behavioral part, that is the actual work of the system, and it is contained in *architecture* unit

In VHDL vocab, a complete VHDL desing which is formed by an entity and a architecture is called *component*.

1.4.1 Entity

The entity contains all input output of our desing. Figure 1.12 shows the parameter and syntax of VHDL entity

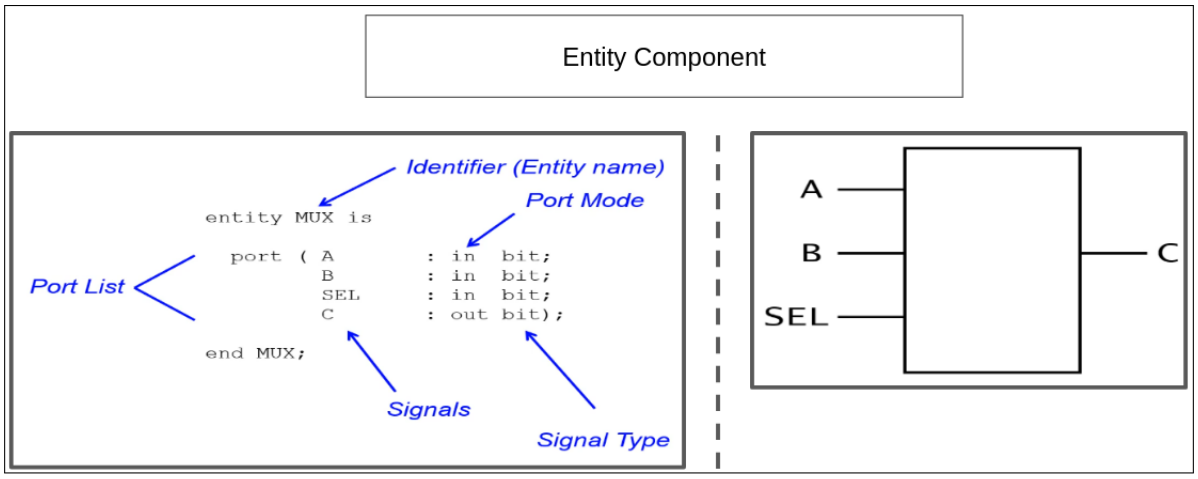


Figure 1.12: VHDL entity

- We begin by the keyword `entity` and its name (`MUX`)
- `port()` contains a list of all input and output
- we type their name (`A`) followed by `:`
- Their mode `in` or `out`
- Their type `bit`
- Finally we use the `end` and the name `MUX`

Some notes:

- In `VHDL`, an entity must be associated with some architecture, where we code our behaviour.
- From a syntax: note that in Figure 1.12 that the last `;` is placed after the parenthesis, and not before

1.4.2 Architecture

The architecture is where we define our functional desing.

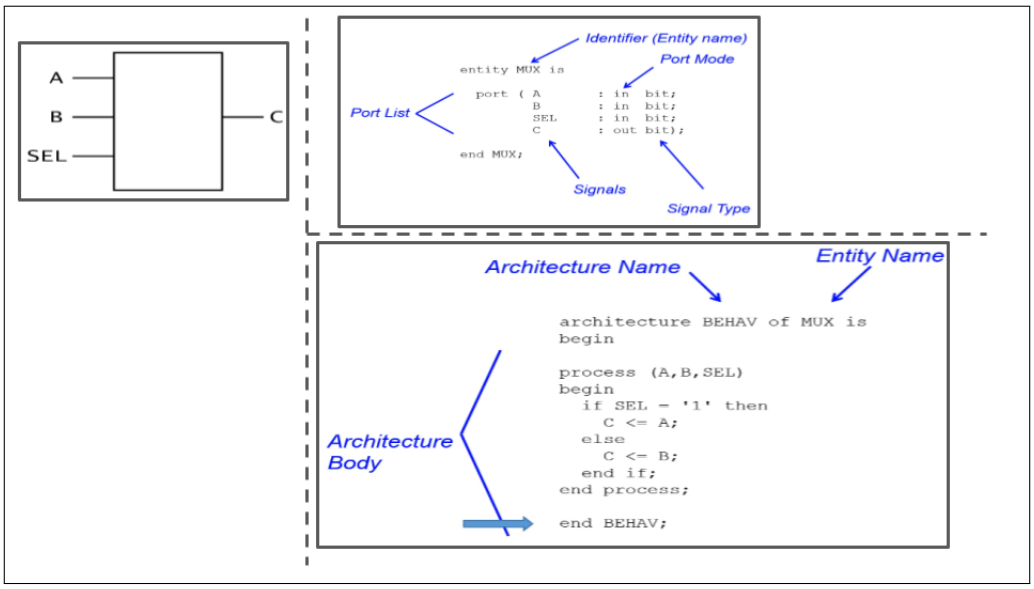


Figure 1.13: VHDL architecture with its corresponding entity

- The 1st line is the *architecture declaration*, where we have the archi name (`BEHAV`) and the associated entity (`MUX`)
- After declaration, we use the keyword `begin`
  - Inside this `begin` block, we put our code, that is the behavioural description of our system
- In the case of a MUX circuit, we put our behavioral description inside `process` block, where we have some sequential statemets, that is statement executed one after another

1.4.3 Internal Signals

Any signal (like a variable) wich is defined in the entity can be used inside the archtiecture. However, sometimes we can use some internal signals, that is signal which are only visibile inside the architecture unit. An example is shown in Figure 1.14.

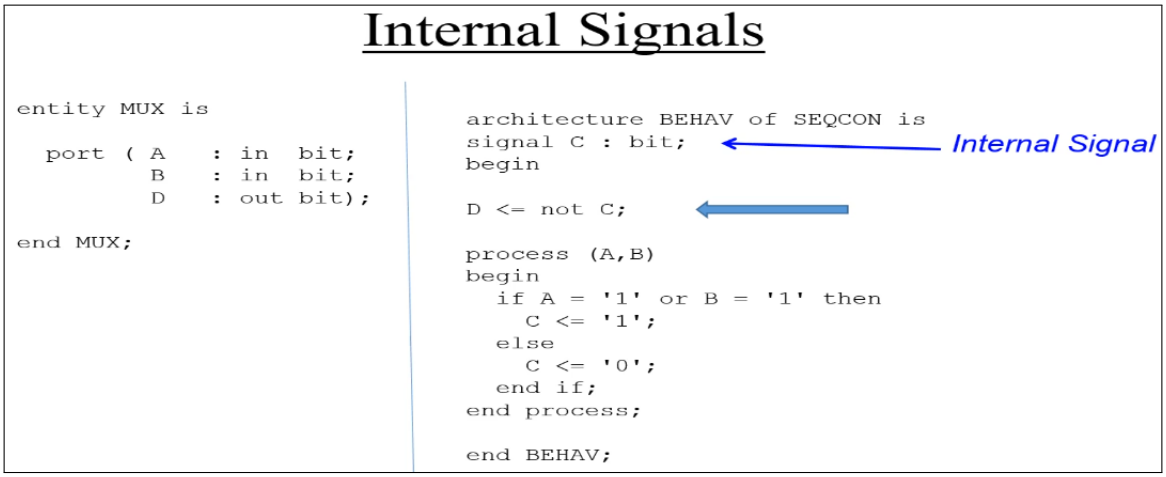


Figure 1.14: Internal signal example

- signal `C` is declared inside the architecture, but in the entity
- It is assigned to `D` by wiring up to a `NOT` gate
- Then it is executed both in parallel with the process

1.4.4 Concurrency

The statement defined outside a process but inside the architecture are executed in parallel with the processes. Figure 1.15 present an example of `AND` implementation

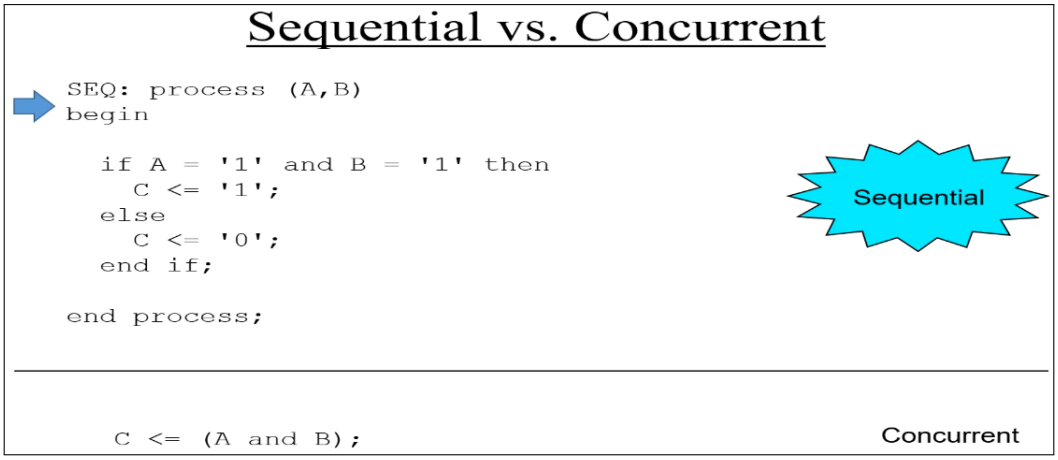


Figure 1.15: and gate implementation using sequential and concurent style

- 1st implementation sequential style using `if-else`
- 2nd style using `and` keyword statement , which is concurrent

As a desingers, we use a combination of both sequential and concurrent statements.

Also note that the order of execution of concurrent statement is not important, since they will all be executed in parallel using the capacity of FPGA.

1.4.5 Process

As we saw processes contains statements that are executed sequentially.
Figure 1.16 presents paramter of VHDL process.

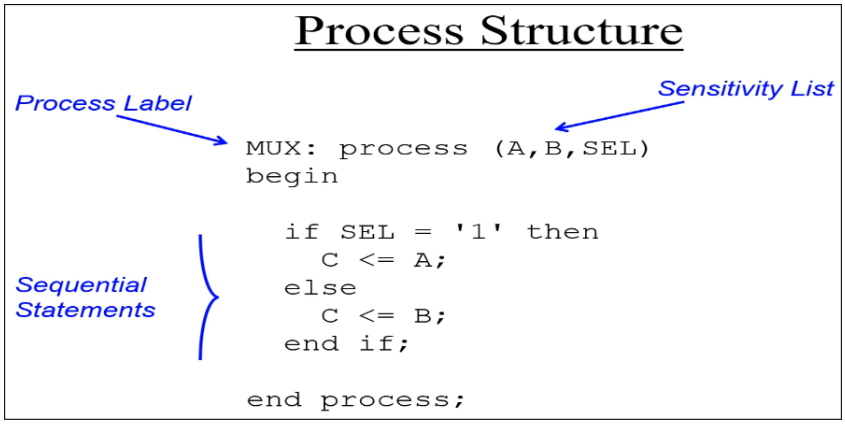


Figure 1.16: VHDL process

- Sensitivity list is used when a process will be executed
  - It is executed when at least 1 of the input is changed

Later we will see why we use sensitivity list in VHDL (section 1.6).

1.5 Some VHDL Examples

In this section we take some example of some VHDL code to get more practice.

- Figure 1.17 present the code of a NOR gate

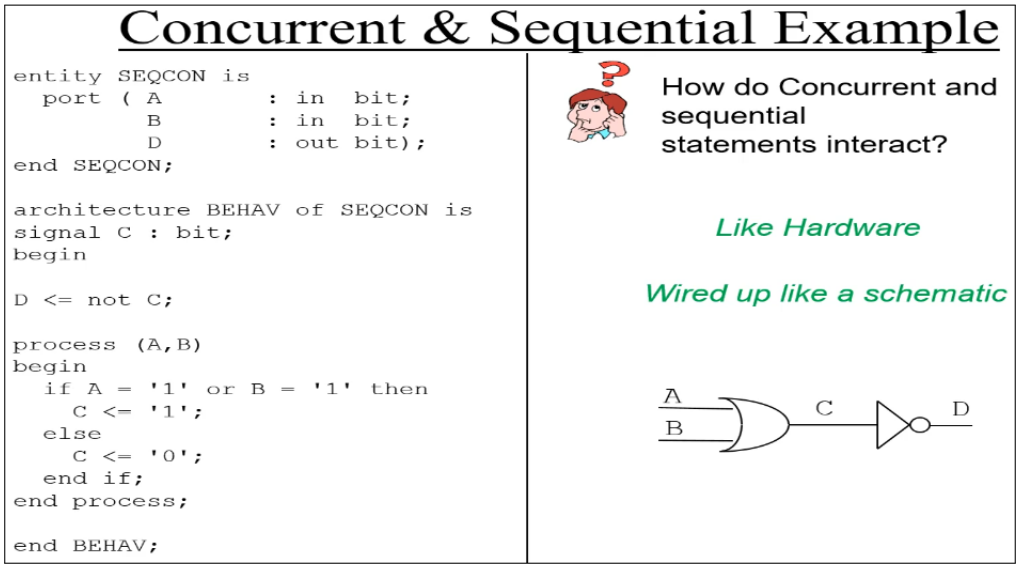


Figure 1.17: VHDL NOR gate

- Like we use wire to connect circuit in schematic, in VHDL we use `signal` to do the connection
- The code of Figure 1.17 present an example of sequential statement interacting with concurrent statement `D <= not C`

- Figure 1.18 presents an example of using 2 same statements, but one outside a process (inside the architecture), and the other inside a process

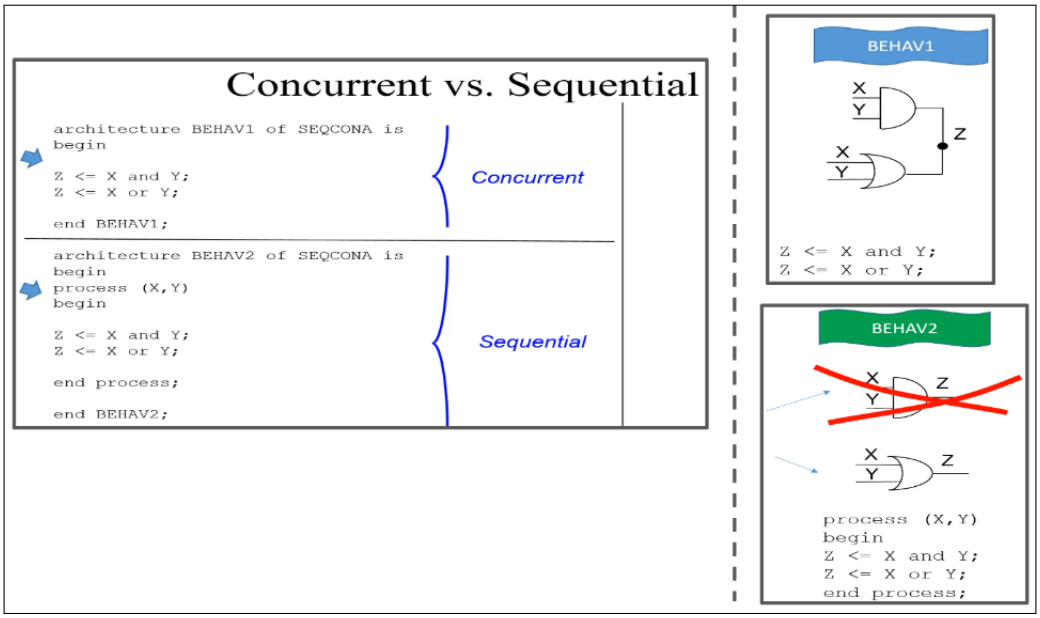


Figure 1.18: VHDL NOR gate

  - This illustrate the difference between sequential and concurrent code
  - In the 1st behaviour, the same set of input (of logic) are driving the same wire. This is isn't good because it causes a *contention*
  - In the 2nd beahviour, it turns out that the first statement get ignored, and only second statment get executed

Actually this one of the `process` rules, which states that:

  - Only last valid assignement to a signal takes place inside a `process`
  - That assignment happens at *the end* of a `process`

1.6 Process Rules

Let's see the `process` rules in this section.

- Process executes when inputs in the sensitivity list change
- Process can only contains sequential statements
- Signal assignment occurs at the *end* `process` statment
  - They don't happen when the statement get executed, they only happen at the end `process`.
- Signal get the last executed assignment

These rules are a little bit of strang at first, but we will revisit them always and need to remember them, since these rules affect our way of designing our code and the sequential statement inside the `process`.

1.7 More about VHDL Langugage

Throughout our progress, we learn more `VHDL` concept and features as needed, so we can grasp concept in a sequential manner.

1.7.1 Bit Vectors

First let's start with signal that are larger than 1 bit, which are `bit_vector`. `bit_vector` are group of bits, or from a programming side array of bits.

An example is shown in Figure 1.19.

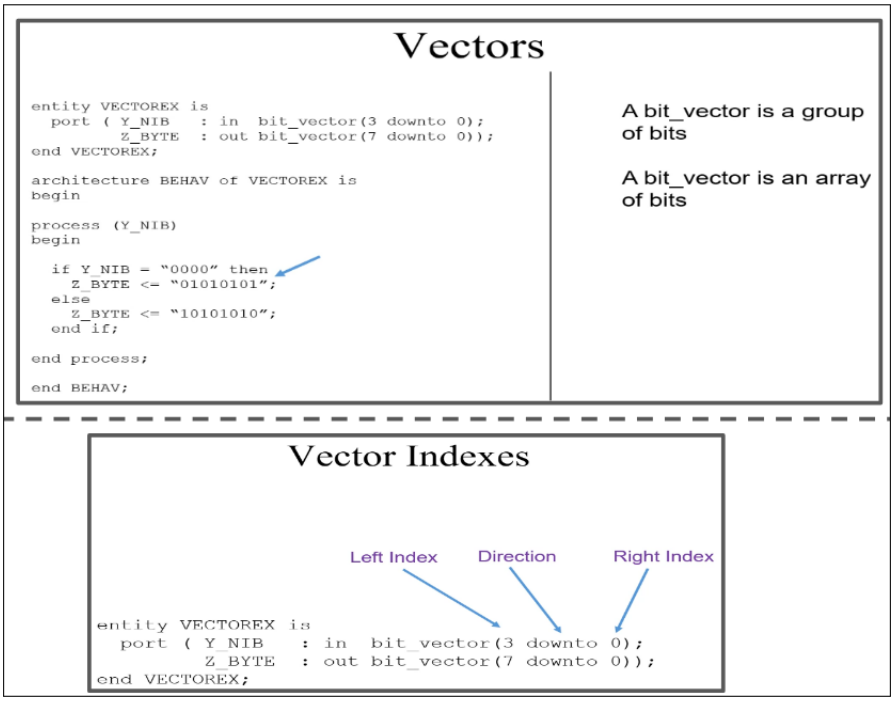


Figure 1.19: Vectors variables in VHDL

- We have `Y_NIB` and `Z_BYTE`, 4 and 8 bit vectors respectevely
- Note for vectors we use double quotation `""`, as opposed to signal which use single quotation `''` for 1 bit case
- For indices: they are going from left to right as shown in the 2nd part of Figure 1.19, with a description in the middle `downto` or `upto`

1.7.2 Bad code: vector mismatch

Nice thing about VHDL it is a strict languafe, it won't let you assign vectors to different size, or of different type. An example of bad code is shown in Figure 1.20.

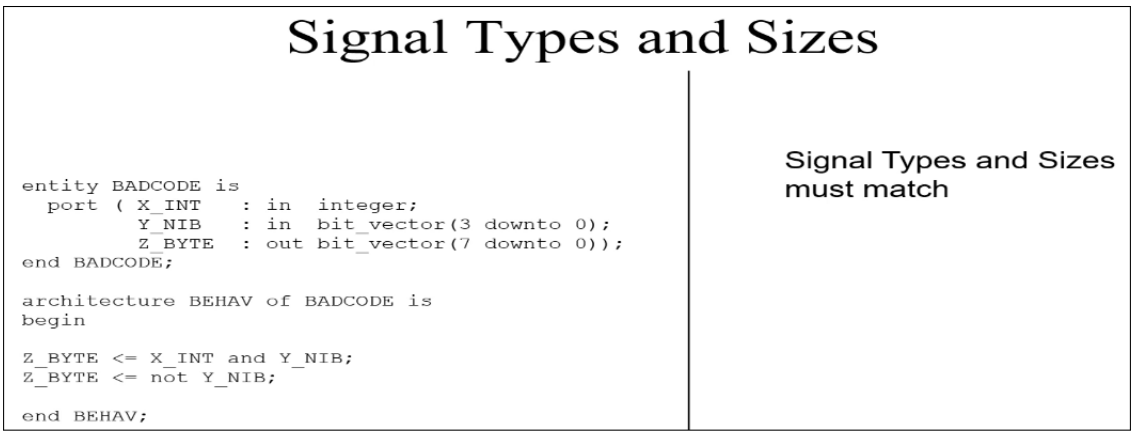


Figure 1.20: Vectors bad code

- We have 2 concurrent statements
- The 1st statement we get a compile time error since we are trying to assign to Z_BYTE from 4 vector, and also a wrong type variable (X_INT is integer)
- The 2nd statement is also wrong since Y_NIB is 4 bit vector, whereas Z_BYTE is 8 bit vector

1.7.3 Code comments

In VHDL comments are insetered after 2 dashes --. An example is shown in Figure 1.21.

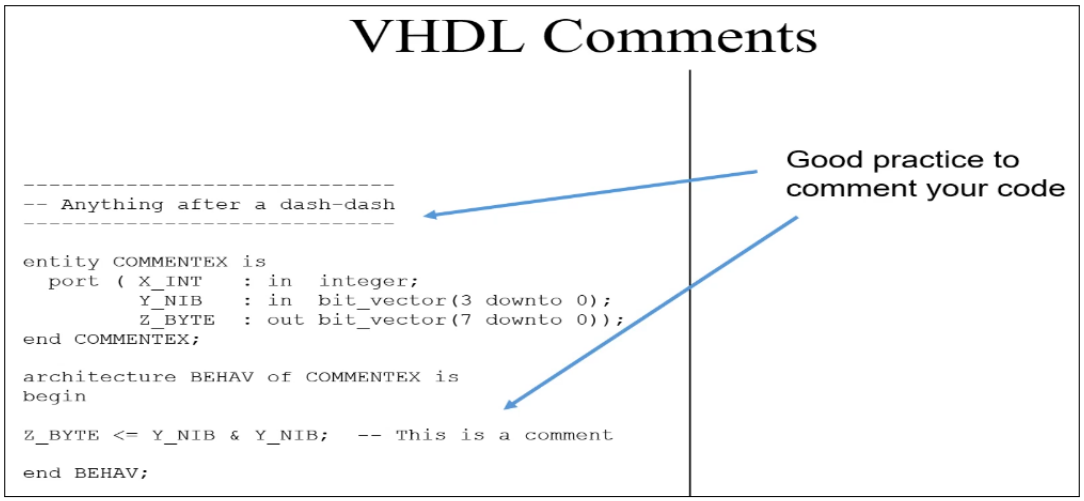


Figure 1.21: VHDL comment

However in VHDL, there isn't a block comment feature as in some other programming language.

1.7.4 VHDL syntax exercice

VHDL is a strict language, and we often need time and practice to get use to it. If we make some mistakes, the compiler gives errors, but it will be hard to find where these error happens, so we need to practice on the syntax and rules.

Given: Figure 1.22 contains a VHDL code with some errors. Try to spot the errors.

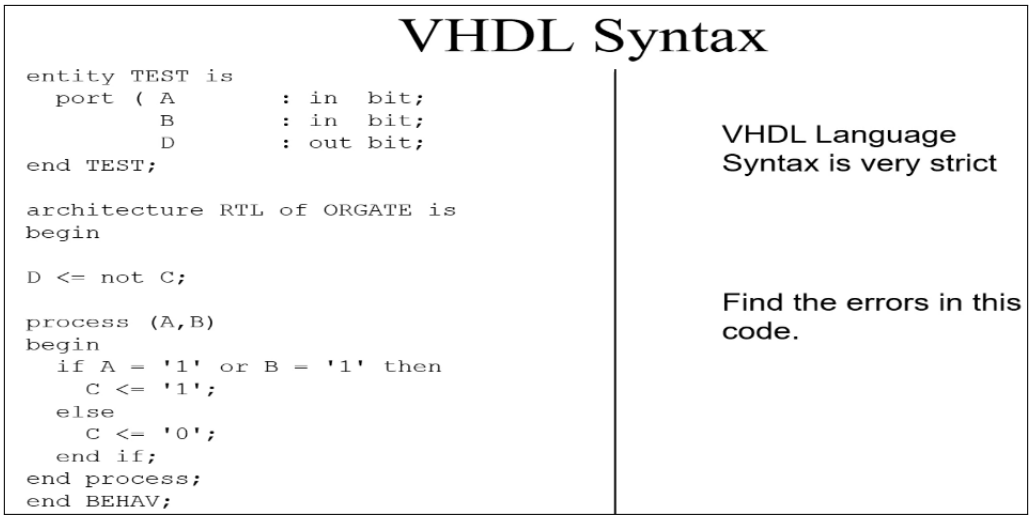


Figure 1.22: VHDL syntax exercice

Solution:

- 1st error in the `entity` declaration, we are missing the last parenthesis before the last semicolone
  - ↔ `D:out bit);`
- 2nd in the `architecture` we are referring to the wrong `entity` name
  - It should be `TEST` instead of `ORGATE`
  - ↔ `architecture RTL of TEST is`
- Signal declaration is missing for variable `C`
  - it should be declared after the `architecture`, and before the 1st `begin` statement
  - ↔ `signal C : bit` (see 1.4.3 and Figure 1.14)
- At last, we are ending with the wrong architecture name
  - it should be `RTL` instead of `BEAHV`
  - ↔ `end RTL`

1.8 Summary Introduction

- Basic idea behind `HDL`: give our desing some desing or behaviour, and let the tools figure out the underlying logic and gates (1.1)
- Desing evolution: transistor $\rightarrow$ gates and logic $\rightarrow$ `HDL` (1.1.1)
- The output of a `VHDL` (or any `HDL`) is a circuit, and not a binary or some executable like known programming languages (1.3)
- Development process of an FPGA (1.2): we have mainly 3 steps

  1. Behavioral simulation: writing code, compiling the code then verify the behaviour using a set of stimilus and visualise them by waveforms
  2. Gate level simulation: which include synthesis verification, then checking if output of logic gates is correct or not
  3. Place and route: make the connections within and between FPGA cells (review 1.1.4 for more details about connections)

Now regarding `VHDL` (1.4):

- From a language desing:
  - Mainly a `VHDL` program consist of 2 parts: entity and architecture
  - Entity: we list input and output, and all their relative information (their types, modes,$\cdots$)
  - Architecture: contains the body of our code, and consist of describing how the system work
- For the architecture:
  - Anything after the architecture delaration is executed in parallel
  - An architecture is a combination of many processes, internal variables declaration, $\cdots$
  - A `VHDL` rule: an architecture must be associated to some entity
- In a `process`
  - all the code is executed sequentially
  - The process should contains a sensitivity list $\leftrightarrow$ `process(A,B)`
  - We say that this process is *sensitiv* to `A` and `B`
- A `VHDL` code usually consist of a combination of sequential and concurrent statement.
  - Order of execution of concurrent statement is not important, since all statement are executed simulatenously, using FPGA (hardware) capacity (1.4.5)

Chapter 2

Sequential and Concurrent VHDL

2.1 Introduction

In this chapter we dive in more details about processes and concurrency in VHDL.
Also we will look at new data types in VHDL: integers and boolean.
Finally we learn how to manipulate vectors.

intro seq an conc

intro seq and conc:

- *To re-write it later*

2.2 Processes in Depth

As stated earlier, VHDL program is composed from ***many processes***. In this section we dive more in depth to processes, and explore 3 different topics:

1. process interaction
2. process rules
3. process behavior

Each one of these concepts is important to desing and understand a VHDLcode.

2.2.1 Processes Interact

Processes (in plural) can interact with each other. The interaction can be:

- in parallel: so processes are executed simultaneously
- sequential: exectued one after another, and usually they depend on each other

The type of interaction (execution) depends on the sensitivity list.

An example is shown in Figure 2.1.

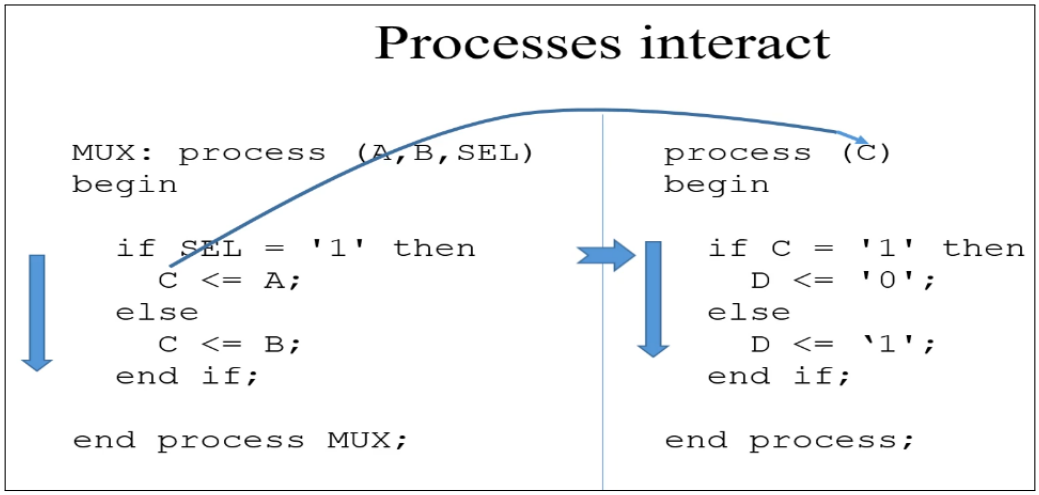


Figure 2.1: Process interaction: sequential execution

- In the 1st process, signal C get assigned to A or B depending on the value of SEL
- Once C is assigned, then the 2nd process get executed, and signal D get assigned

We can think that a VHDL program consist of many processes interacting with each other, as shown in Figure 2.2.

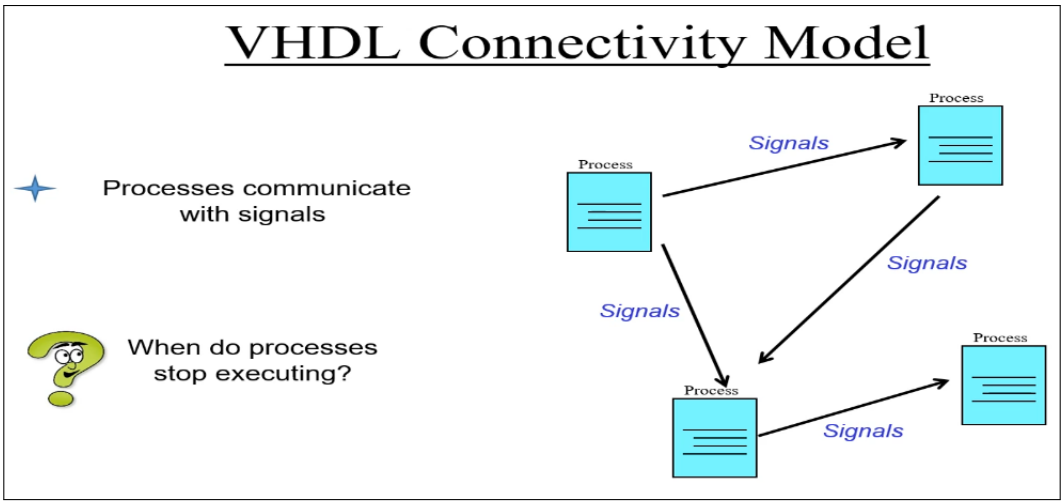


Figure 2.2: General view for process interaction

Processes interacting with each other via signal (variables) assignment. The variables presented in process A are in the sensitivity list of process B

2.2.2 Warning: Asynchronous Feedback

Now we take an example shown in Figure 2.3, wich can cause some problems.

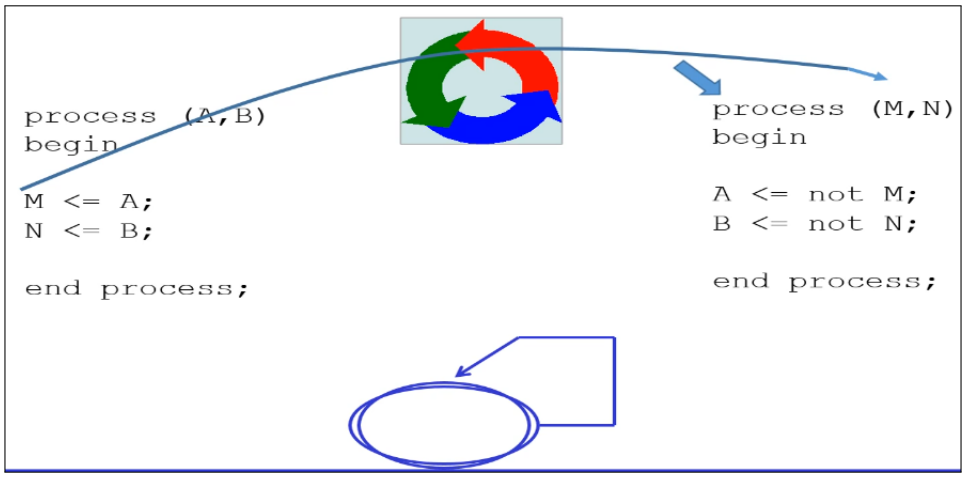


Figure 2.3: Asynchronous dilemma

- 1st process (sensitive to A and B): we have assignment of M and N
- 2nd process we have signal assignment of A and B
  - This cause the execution of the 1st process because it is sensitiv to A and B

This type of signal assignment cause a loop, and will result in failur in simulation. We should be aware of such assignment and try to avoid them.

2.2.3 Process Rules

Now let's review rules of process.

1. Sensitivity list

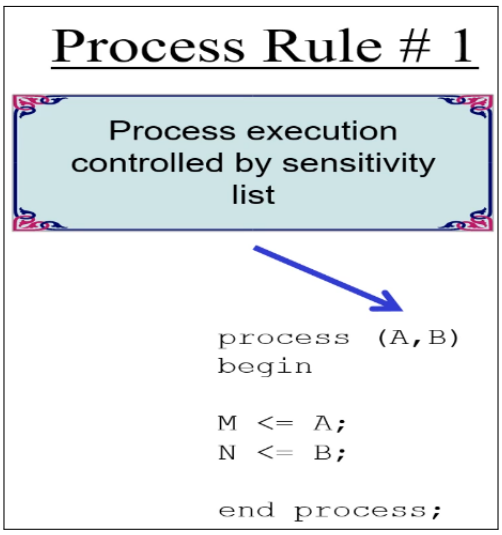


Figure 2.4: Rule 1: Sensitivity list

- A `process` get executed if 1 or all the inputs on the sensitivity list change
- That's why we need to ***include all the inputs in the sensitivity list***, so the process get exectuted correctly whenver an input change

2. Signal assignment

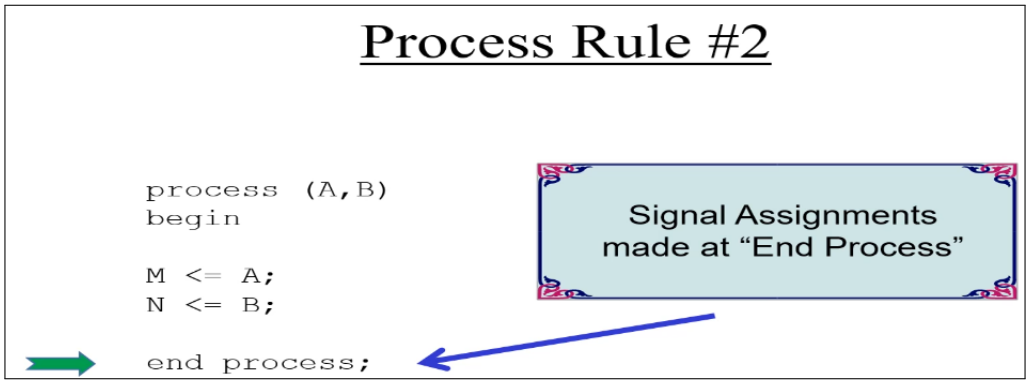


Figure 2.5: Rule 2: signal assignment

- This is the strangest one: signal are begin assigned ***only at the end of a process***
- So at the line of execution where `M<=A`, `M` won't be assigned till the end of the `process`
- This mean we won't be able to use `M` for some intermediate computation
- If we want to do such computation, we use ***variables***. We learn about them later.

3. Multiple assignment

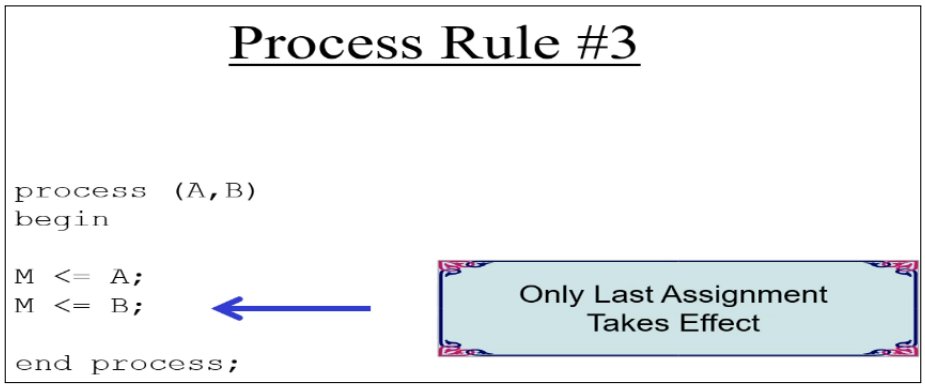


Figure 2.6: Rule 3: Multiple assignment

- If we make multiple assignment to the ***same signal***, only the last one get executed
- So `M <= A;` get ignored
- And `M <= B;` get executed at the ***end*** of process (recall rule 2)

2.2.4 Process Behavior

Now we learned process rules in 2.2.3, let's take an example about the execution of a process, shown in Figure 2.7.

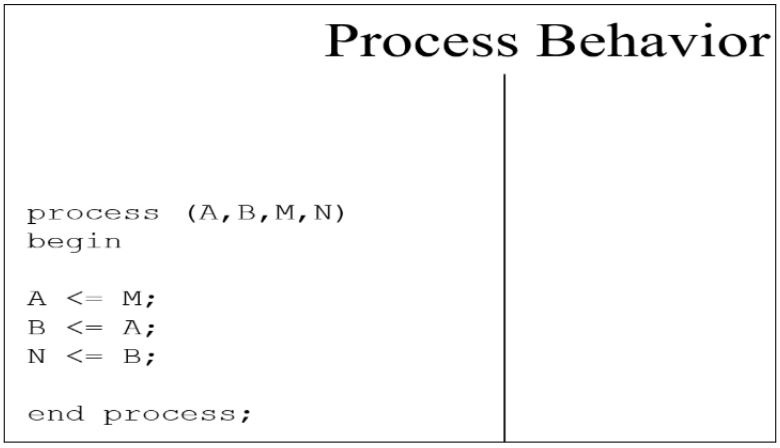


Figure 2.7: Process Behavior

Question: what are the final values of A,B and N, if this process is executed for 3 times ?

Answer: values are shown in Figure 2.8.

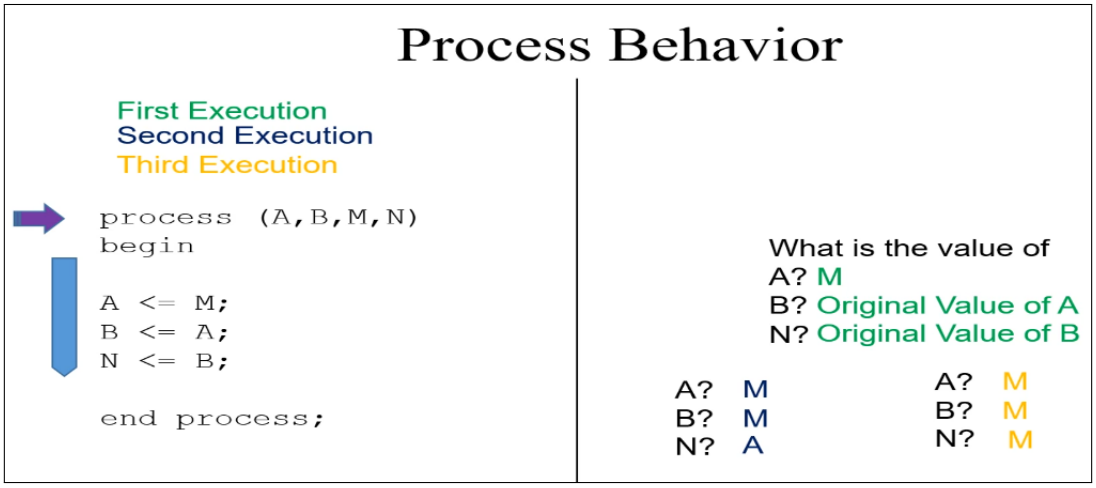


Figure 2.8: Process Behavior

2.3 Data Types

So far we dealt with bit and bit vectors data types (**??**), we continue explore other data types in VHDL such as integers and boolean in this section.

2.3.1 Integers

An example is shown in Figure 2.9.

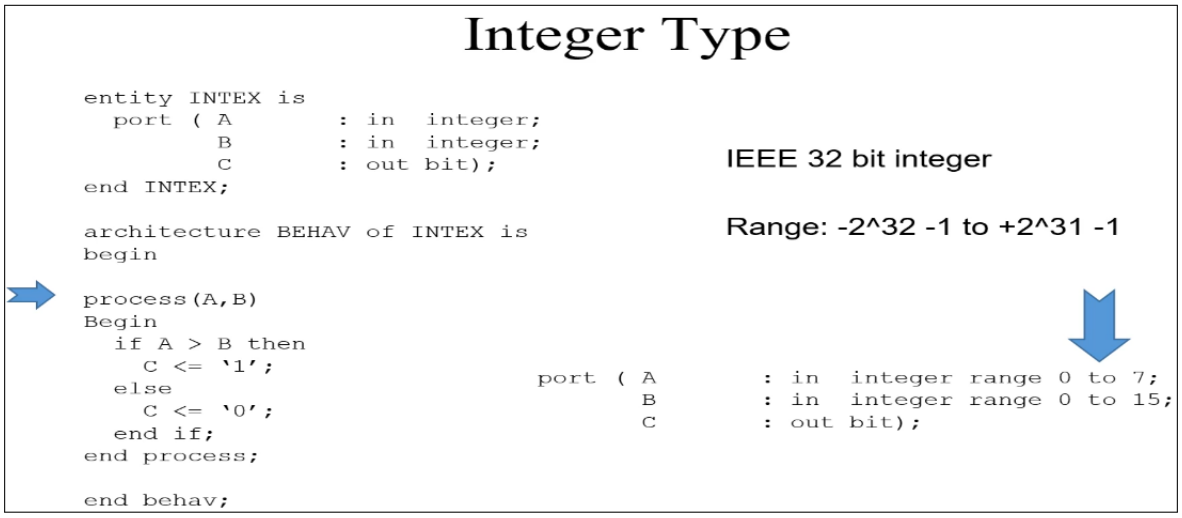


Figure 2.9: Integer data type in VHDL

- In VHDL, and since almost we don't need huge range of values of integers in our application, we can specify the range of intergers we need as shown at the right bottom of Figure 2.9

2.3.2 Boolean

Also VHDL support boolean as shown in Figure 2.10.

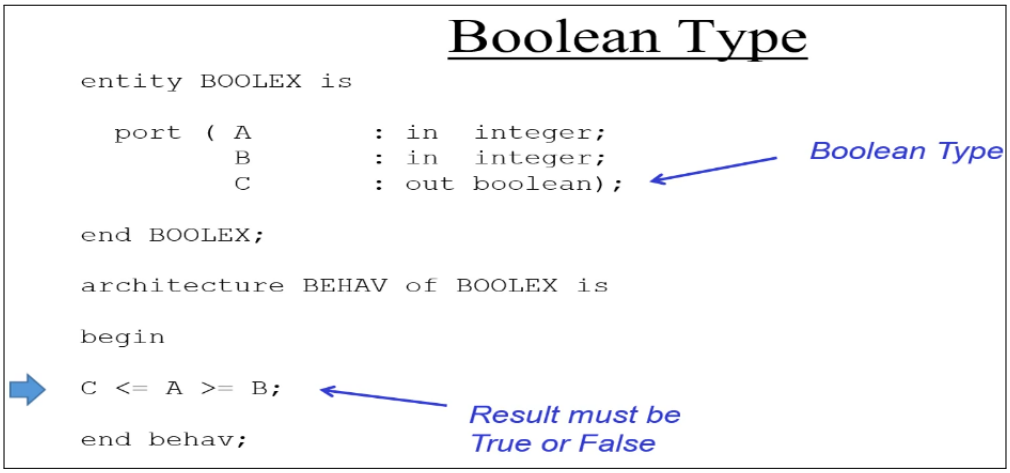


Figure 2.10: Boolean data type in VHDL

- In the example, `C` get assigned to 0 or 1 whether the condition `A >= B` is true or not

2.3.3 Litteral Assignment

Sometimes we need to do an assignment to signal, and depending on the type of signal, the assignment way may differ. An example is shown in Figure 2.11.

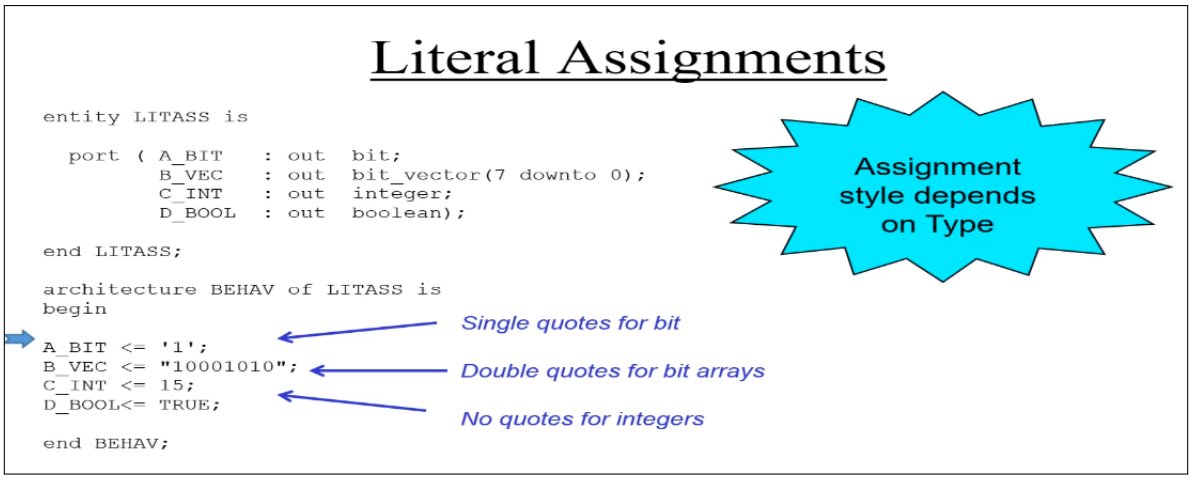


Figure 2.11: Signal assignment

- signal of types bit need quotation
  - signle for bit, and double for bit array
- Integer and boolean don't need quotation

2.3.4 Vector Comparison

When comparing 2 vectors in VHDL, we should pay attention that the vector size must mismatch, that is they both have same number of bits, otherwise we may have some unlogical results. An example is shown in Figure 2.12.

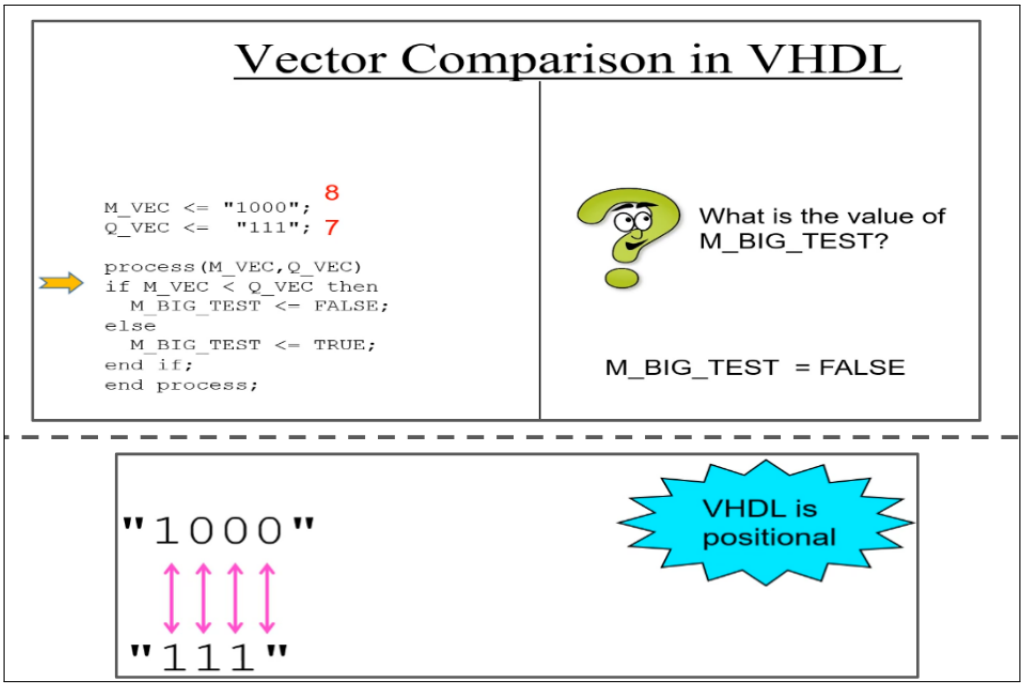


Figure 2.12: Vector comparison

- In VHDL, comparison is made from ***left to right***, and ***bit by bit*** basis
- That's why we can't interpret the numbers M-VEC and Q-VEC as 7 and 8

2.3.5 Vector Aggregation and Concatenation

Finally we see 2 famous operation with vectors: cocncatenation and aggregration,shown in Figure 2.13

```
entity CONCEX is
  port ( A_BIT   : in  bit;
         B_BIT   : in  bit;
         C_BIT   : in  bit;
         X_VEC   : out bit_vector(3 downto 0);
         Y_VEC   : out bit_vector(7 downto 0));
end CONCEX;

architecture BEHAV of CONCEX is
begin

X_VEC <= (A_BIT , B_BIT , C_BIT , '0');

Y_VEC <= ( 1 => A_BIT , 5 => B_BIT , 6 => '1', others => '0');

end BEHAV;

architecture BEHAV of CONCEX is
begin

X_VEC <= A_BIT & B_BIT & C_BIT & '0';

Y_VEC <= "01" & B_BIT & "000" & A_BIT & '0';

end BEHAV;
```

Figure 2.13: Vector aggregation and concatenation

- We have 2 different architecture:
  - 1 for aggregation, which uses `(A,B,...)`
  - 1 for concatenation, uses `&`
- for the aggregtation example concerning `Y-VEC`: this is called ***explicit aggregate***
  - that is index 1 is getting assigned to `A-bit`,index 5 to `B-bit`, and so on

2.4 Sequential and Concurrent Statement

Now we more about processes, and the concept that inside a process all of the statements are sequential, and outside the process the statements are concurrent, we explore in this section more about the used statements in our desing.

2.4.1 If-else

`if-else` structure are used allot in our desing, and especially the nested `if-else`. An example is shown in Figure 2.14.

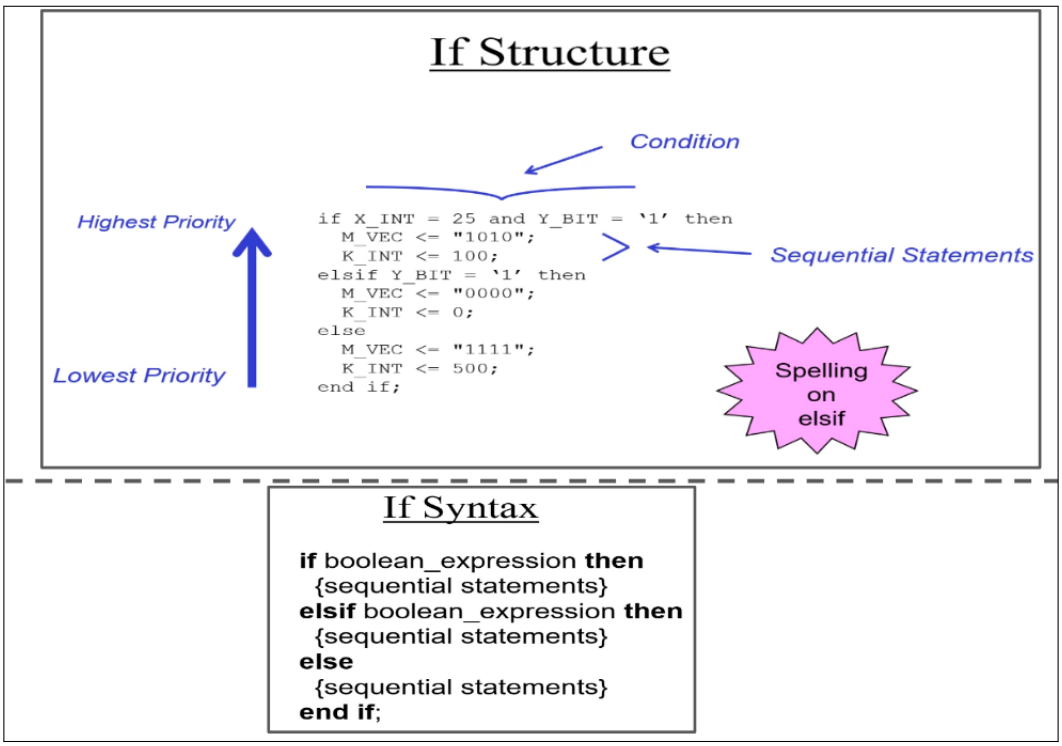


Figure 2.14: If else in VHDL

- Each `if` or `elsif` is followed by some condition with some ***priority***
- Priority value goes from high → low when walking down an `if-else` chain:
  - 1st `if` has highest priority
  - Last `if` has the lowest one

As implementation, `if-else` are synthesised into MUX circuits:

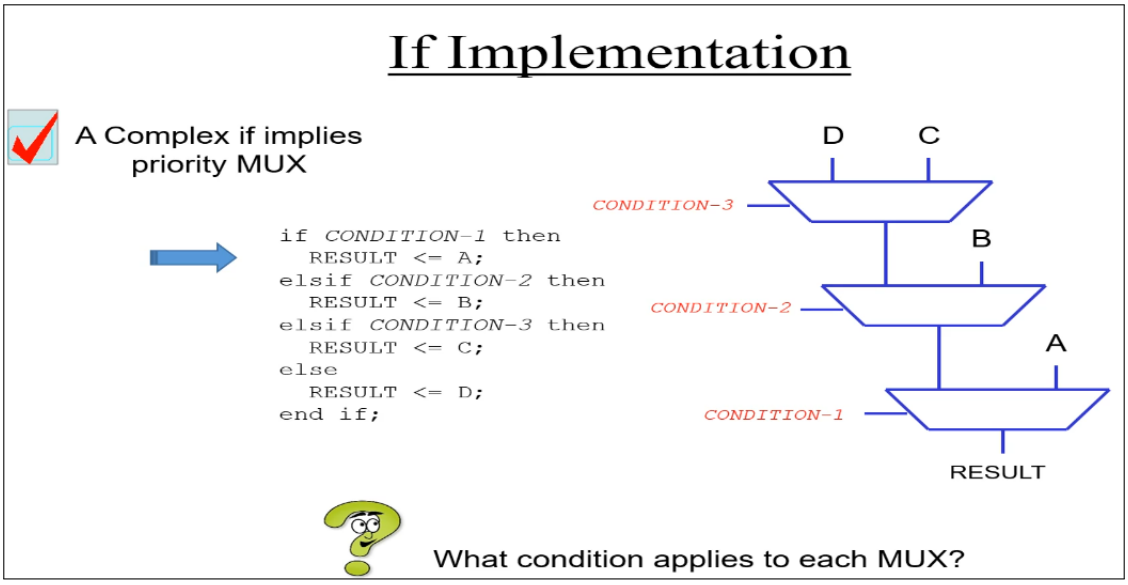


Figure 2.15: If else circuit

- If we have nested `if-else`, a priority MUX is synthesised as shown in Figure 2.15.

To complete the `if` example, conside the example shown in Figure 2.16.

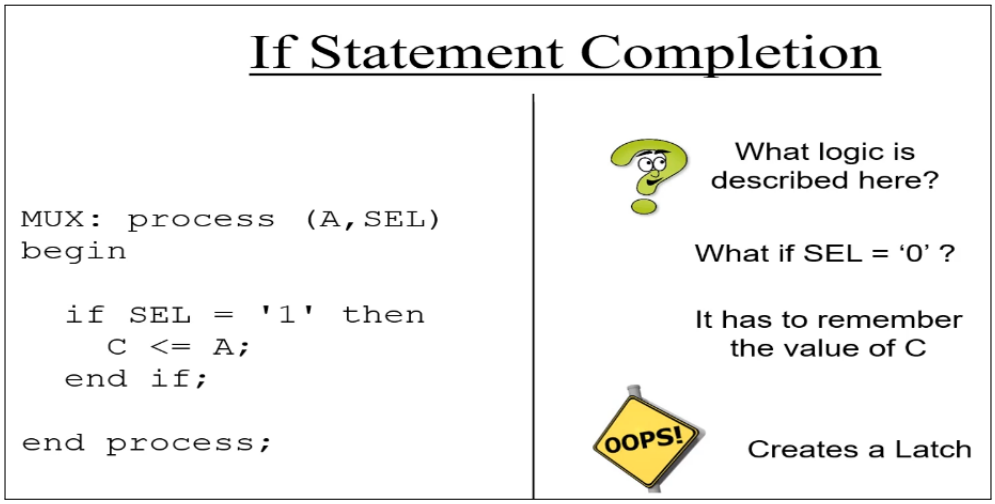


Figure 2.16: Simple if statment

- The problem for the `if` statment shown in Figure 2.16, we don't have a value for `SEL`
- So when `SEL = 0`, the hardware is obliged to create a latch to remember the value of `C`
- Usually in a desing we need to avoid creating latches, so we try to implement a complete `if` statements

nd latche

if and latche:

- *to see later why this is bad to create latches*

In Figure 2.17, we have 2 way of implementation of a complete `if` statements to prevent latch synthesis

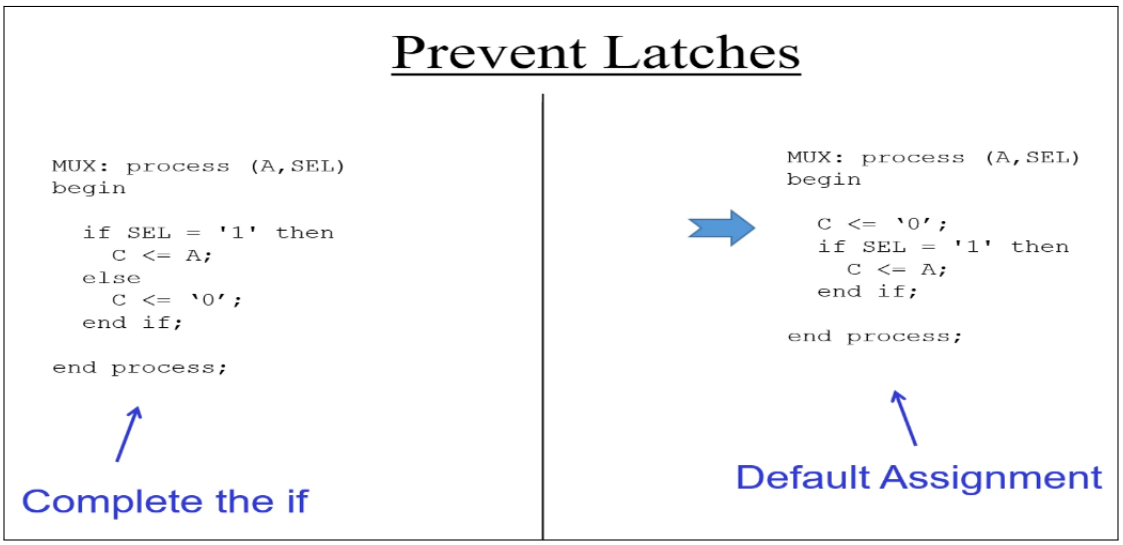


Figure 2.17: Simple if statment: preventing latches

- The 1st one is the normal to complete the `if`: we put a value of `C` inside `else` when we have no value of `SEL`
- The 2nd one is done using a ***default assignment***: we start by assigning `C` a value of `0` before even entering the `if` statement.
  - So when `SEL` is not `1`, the last value of `C` is `0`

The 2 methods implement the same logic, and prevent latches.

2.4.2 Case

It is true that in `VHDL` we use allot `if`, but also we can use other types of selection statement, such as the `case`. An example is shown in Figure 2.18

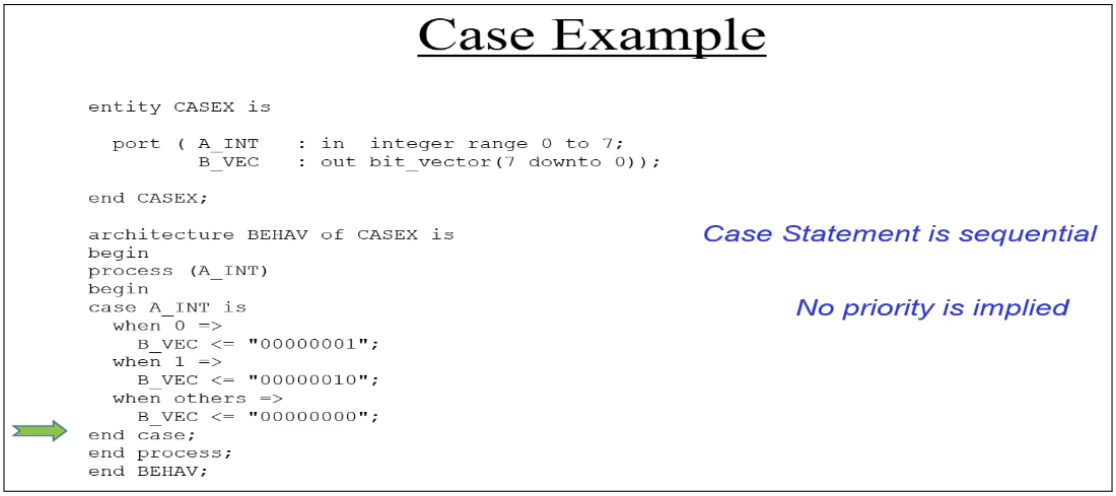


Figure 2.18: Case statement in VHDL

- Unlike the `if`, the `case` has no priority
- So a flat MUX is used to synthesise the hardware

Also be aware that in the conditions, that is `when .. to ..`, we can't use bit vectors (as shown in Figure 2.19), we use only normal integer value or some corresponding enumerated type.

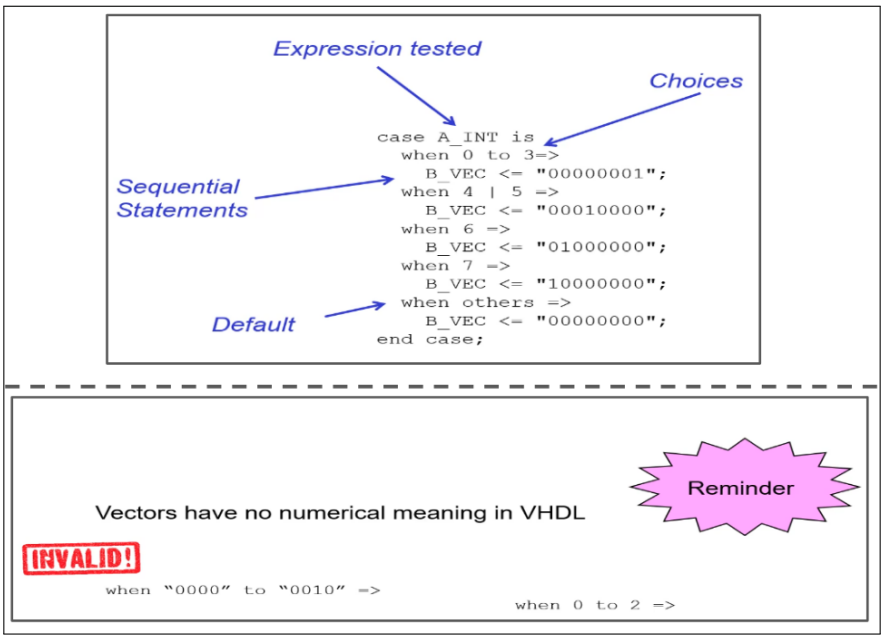


Figure 2.19: Case statement in VHDL: valid numerical types

if vs case

So `if` has priority and `case` has no priorty. Their MUX are shown in Figure 2.20.

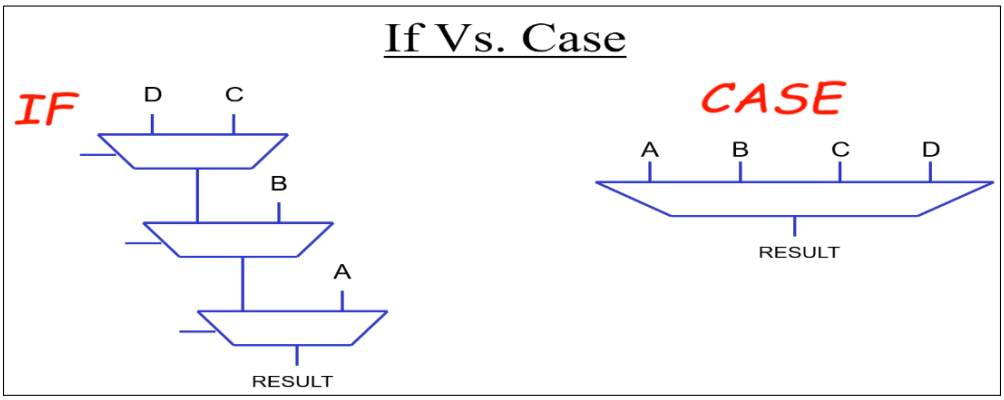


Figure 2.20: if vs case

- The MUX of a `case` has fewer gates
- So if we have no priorty, we should use `case` over `if`

Also we can do some nesting between `if` and `case`, and this will result in a complex logic circuit (MUX of MUX).

2.4.3 for loop

Now we see `for` loop, shown in Figure 2.21.

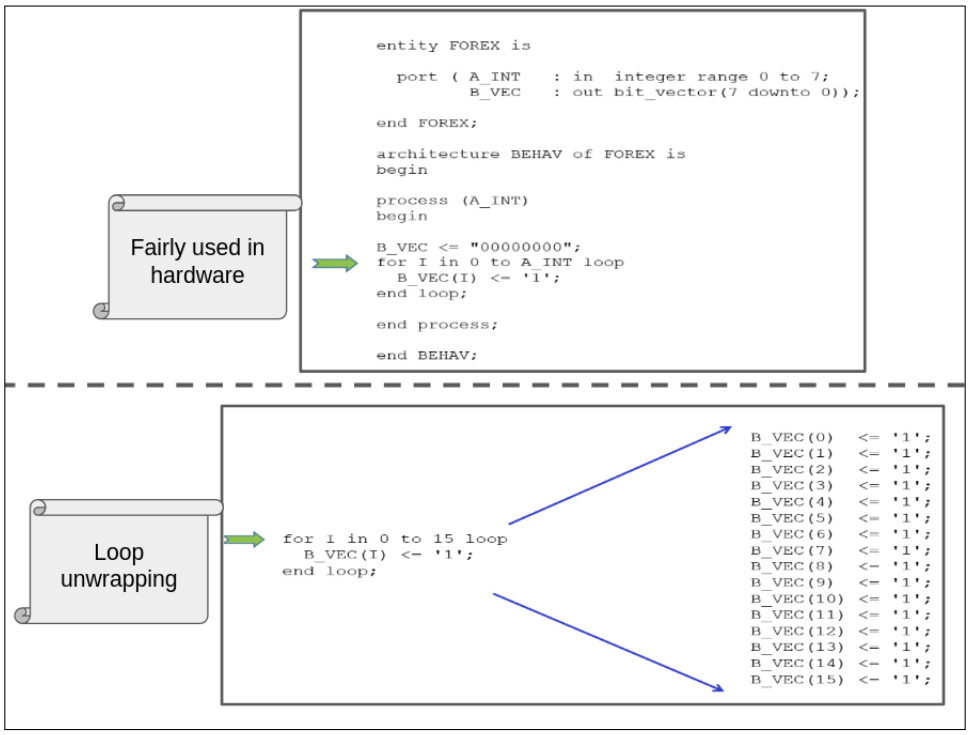


Figure 2.21: for loop

- Here the code is flipping `B-VEC` bits from $0 \rightarrow 1$
- The `for` loop range goes from $0 \rightarrow$ `A-INT`
- This for loop implement the power function $2^{\text{A-INT}-1}$

Notes about **for** loop:

- `for` are rarely used in hardware, and they are used in specialised cases
- the circuit or the synthesiser don't implement a `for` loop, instead it will implement the equivalent sequential statements, as shown in the 2nd part of Figure 2.21
- The `for` loop are generally synthesisable, that is we can use them in a synthesised code
  - but other types of loops (`while()`) we can't use them in a synthesisable code, we can only use them in some verification code

We can also break or jump over an iteration inside a `for` loop, using `next` keyword, as shown in Figure 2.22.

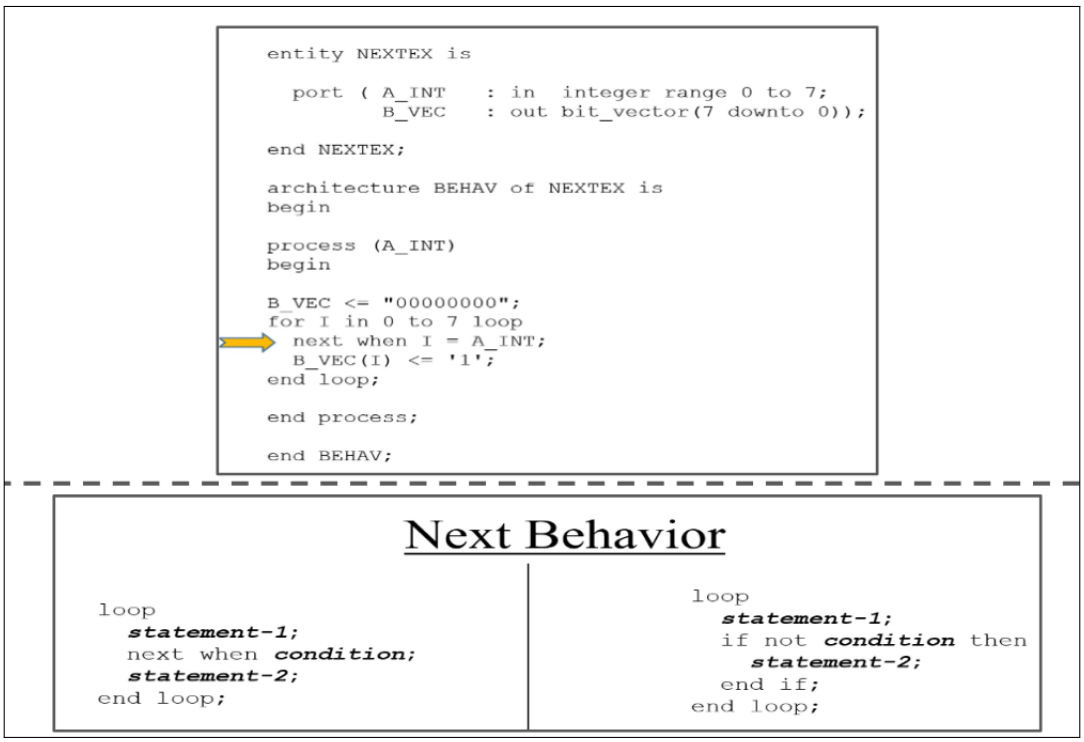


Figure 2.22: Jumping over an iteration

- the `next` also can be implemented an equivalent `if not` as shown in the bottom of Figure 2.22

2.5 Concurrency

Now after we have seen different sequential statements, we move to conccurrent one. Recall that concurrency (seen for the 1st time in 1.4.4) are the satement which happens outside a process, but inside architecture. Their statement runs in parallel, and in continous manner. They require no sensitivity list.

In this section we see different types of concurrent statements.

2.5.1 Conditional Signal Assignment

The 1st one is conditional signal assignment using `when .. else`. An example is shown in Figure 2.23.

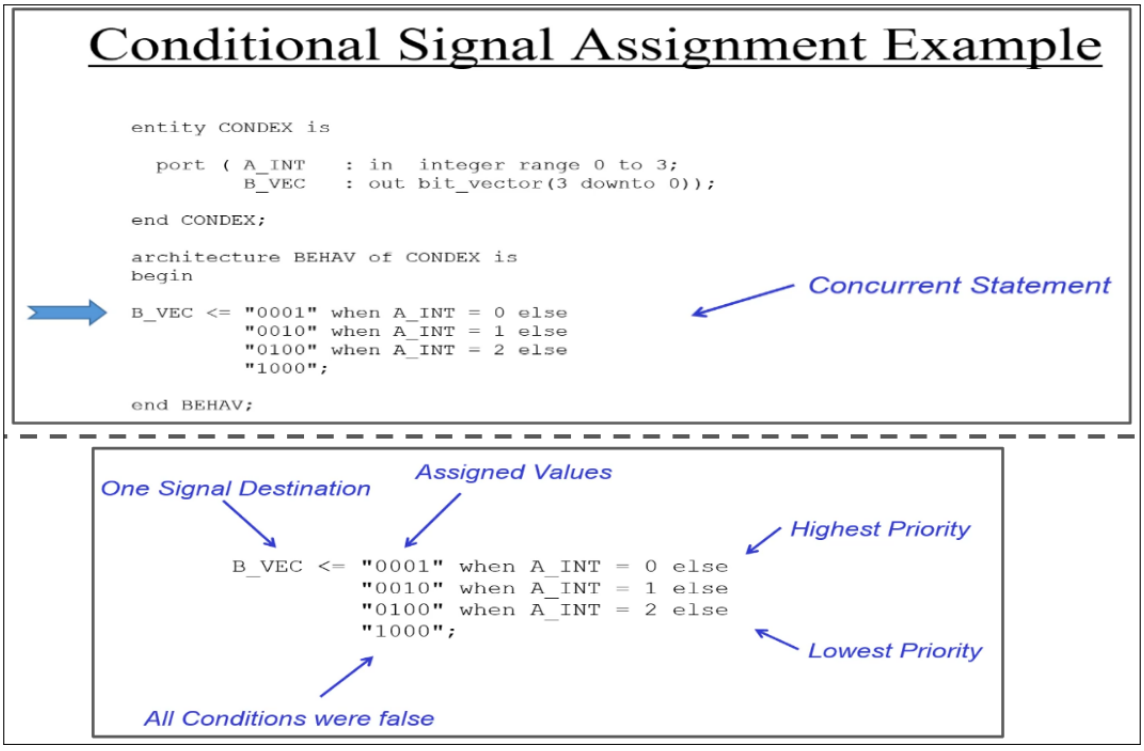


Figure 2.23: Conditional signal assignment

Conditional signal assignment can be done using sequential `if .. else` statment.

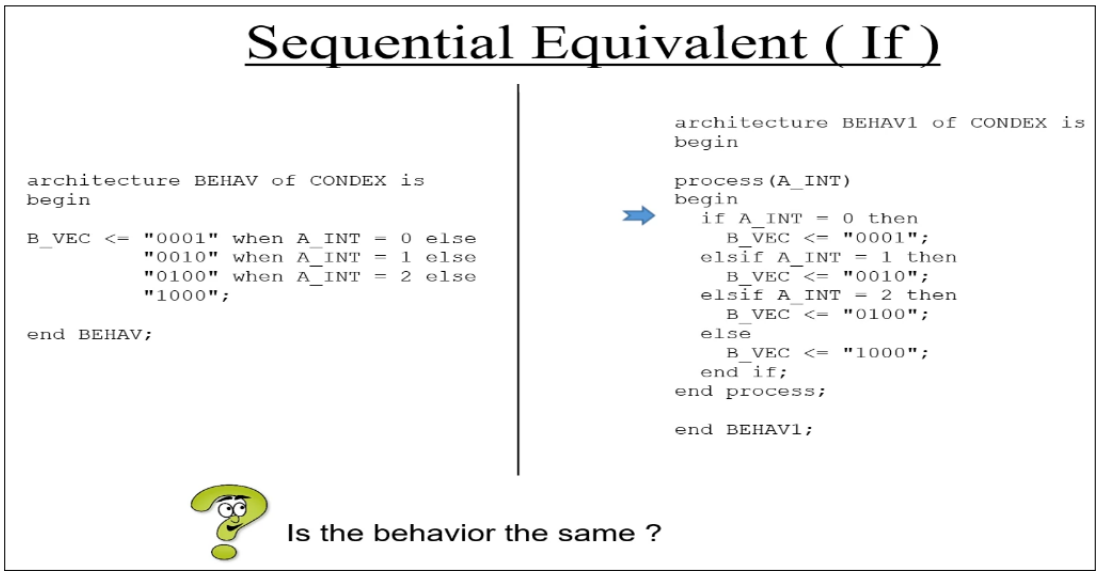


Figure 2.24: Conditional signal assignment equivalency with if statement

when and if equivalency

when and if equivalency:

- *To see later which behaviour that gives, and which are the optimum*

2.5.2 Selected Signal Assignment

The 2nd concurrent statment is the selected signal assignment using `with .. select`. An example is shown in Figure 2.25.

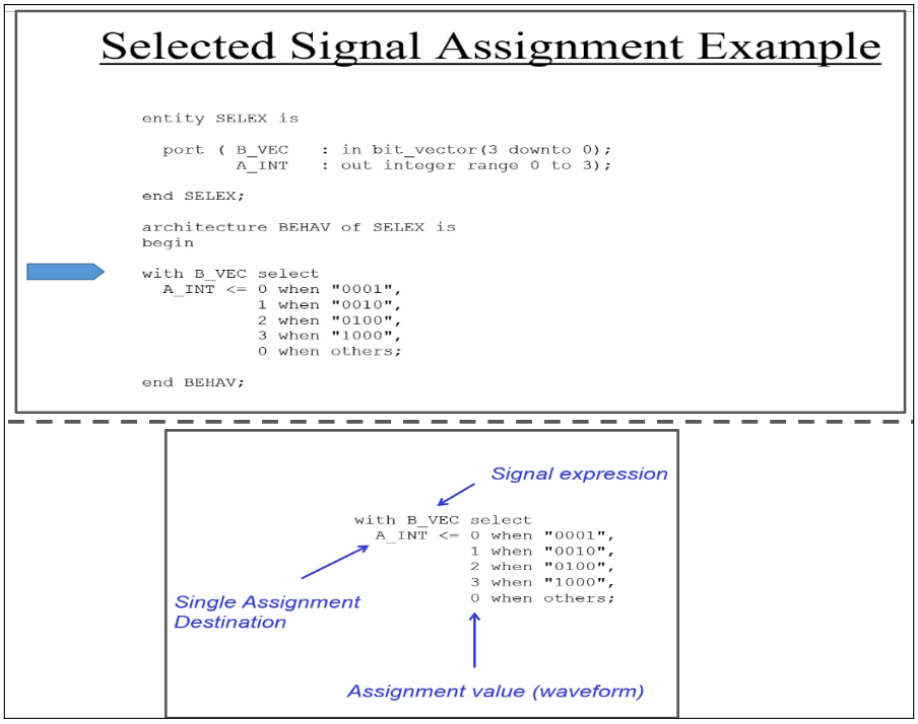


Figure 2.25: Selected signal assignment

As in 2.5.1, the `with .. select` is equivalent to `case` statement, as shown in Figure 2.26.

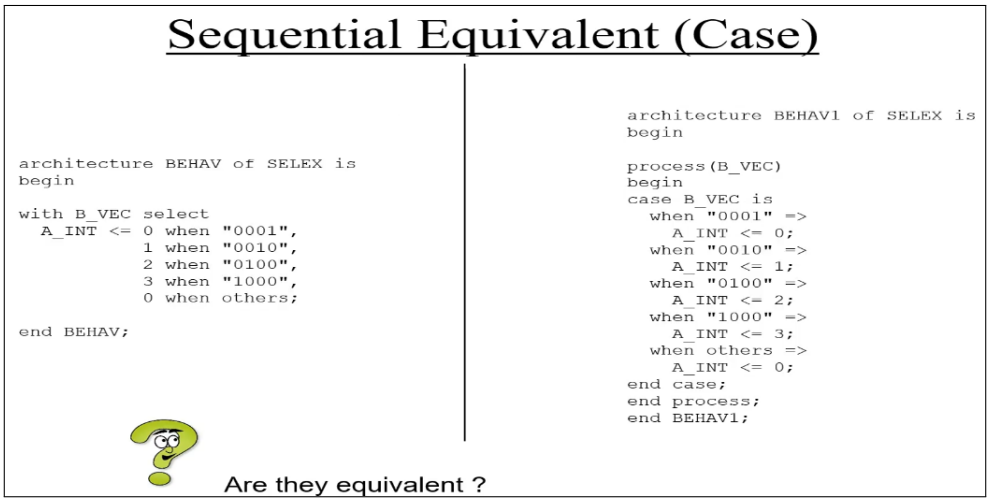


Figure 2.26: Selected signal assignment equivalency using case statment

2.6 VHDL Hierarchy

Until now, we have seen the basic desing of VHDL, that is a VHDL module is composed from minimum 2 parts: entity and architecture (1.4).

At some point our work will get bigger and larger, and the easier way is to decompose our VHDL program into a ***set of separate modules***, and the recombine these different modules into a larger desing.

Let's make the analogy between the program side of VHDL and the hierarchy we make in schematics.
We choose schamtics because VHDL is describing hardware, and not typical software.

In schematics, hierarchy was always done by creating some symbols that groups some logic, like the MUX example seen in 1.1: the shcematic is the MUX symbol, and underneath the symbol is the set of logic gates which formes the MUX.
Another example of shematic hierarchy is shown in Figure 2.27.

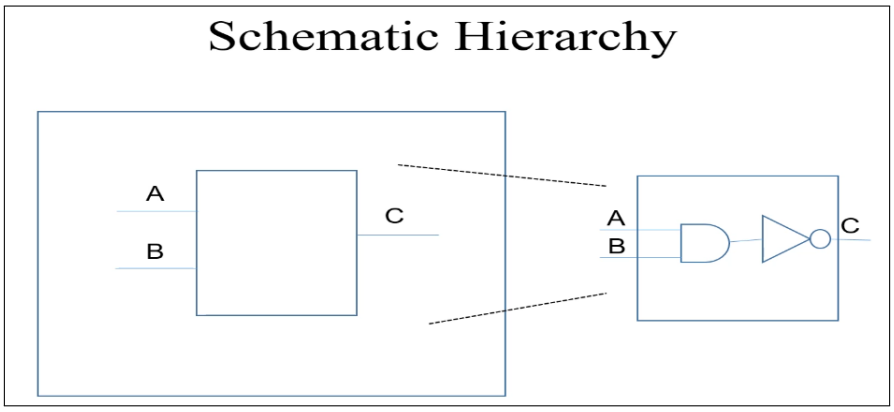


Figure 2.27: Schematic hierarchy example

- In the 1st block we have some symbol with some input signals A and B
- But the symbol is acting like a black box, we don't know the underlying logic beneath it
- So we need to open another page of the schematic to see what is inside

Now in schematic, hierarchy is done via symbols and graphics. VHDL is a set of text files, so we don't have graphics.
The way we do hierarchy in VHDL is as follows:

- 1st step in schematic was symbol creation ↔ in VHDL we declare a component
- 2nd step in schmatic we place the symbol into another page ↔ in VHDL this called componend instantiation
- In schamtics, symbols get connected together with wires ↔ in VHDL components get connected together with signals

2.6.1 Example: Parity System

System Presentation

Let's take some practical example about some system we want to implement. The system is shown in Figure 2.28.

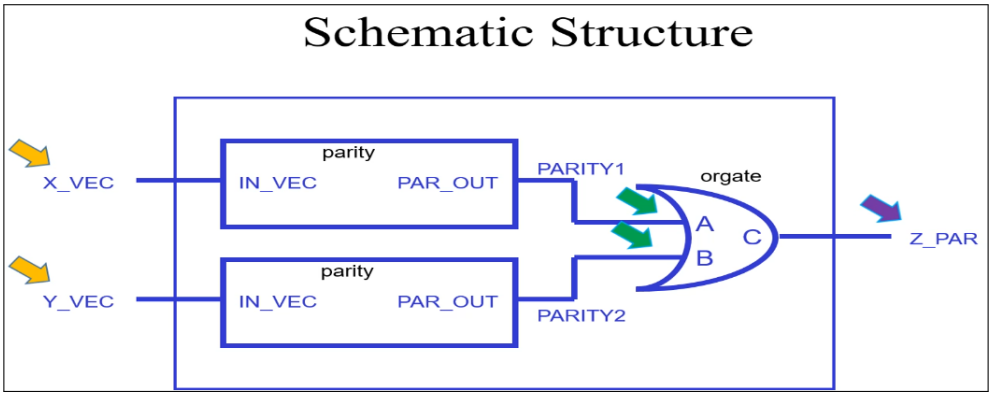


Figure 2.28: Parity Schematic Example

- The system computes the parity of some input vector
- The schematic is composed from 3 symbols wired togther
  - 1 symbol called parity (we have 2 of them)
  - and 1 `OR` gate symbol
- The signal of parity symbol is connected to *to level signal* `X-vec` and `Y-vec`

Component Creation

So what we can see from the example of Figure 2.28

- The lower level in a schematic are the symbols
- In `VHDL`, these lower level are called ***components***
  - The component are nothing more than an entity and an architecture

Figure 2.29 presents this analogy between parity symbol and its VHDL component.

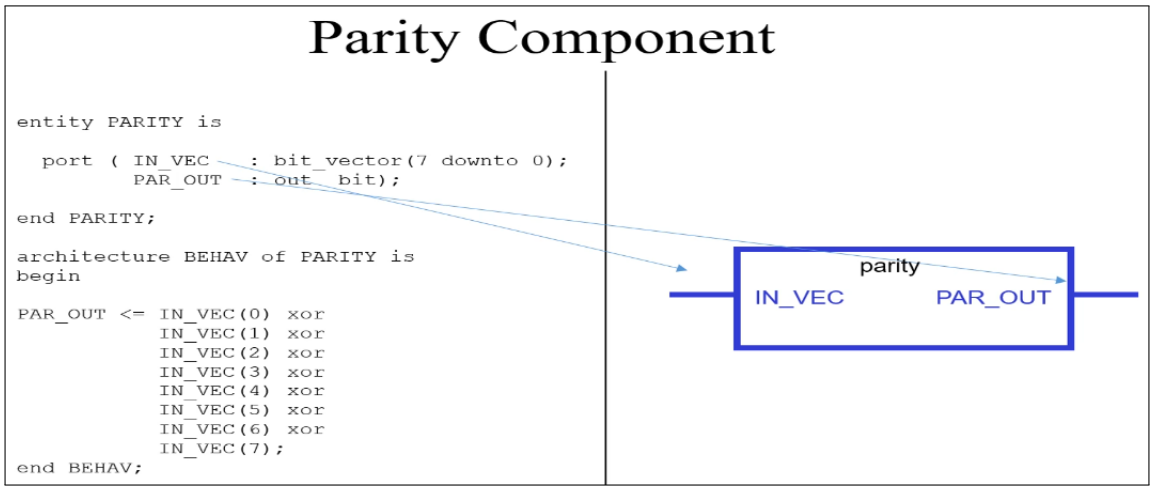


Figure 2.29: Parity symbol, and its VHDL component

- To compute the parity of a vector, we `xor` all of its bits
- this done using conccurrent statment

Recall that the parity component will be kept in a spearate file.

Figure 2.30 presents the same concept with `OR` symbol.

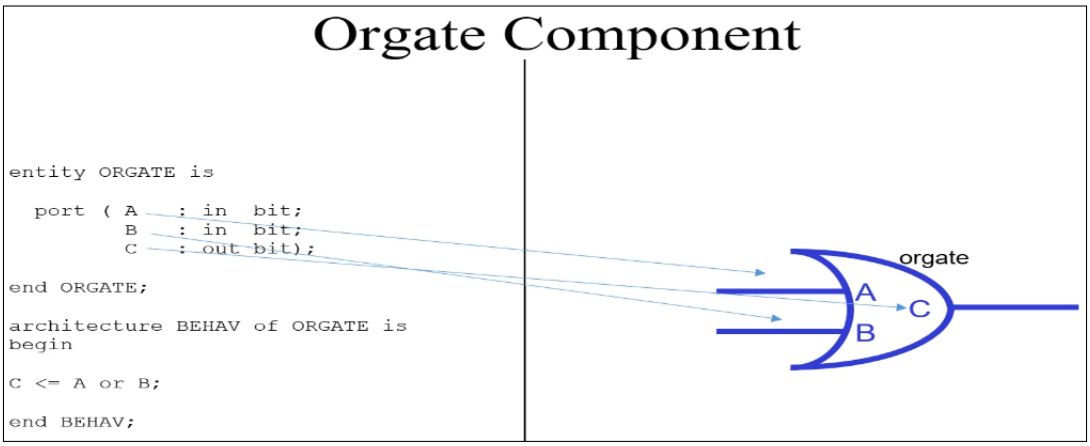


Figure 2.30: OR symbol, and its VHDL component

Component wiring

After creating each VHDL component for each symbol in our schematic example, we need to connect these components.
In schematic, symbols are connected to top level signals using wires as shown in Figure 2.31.

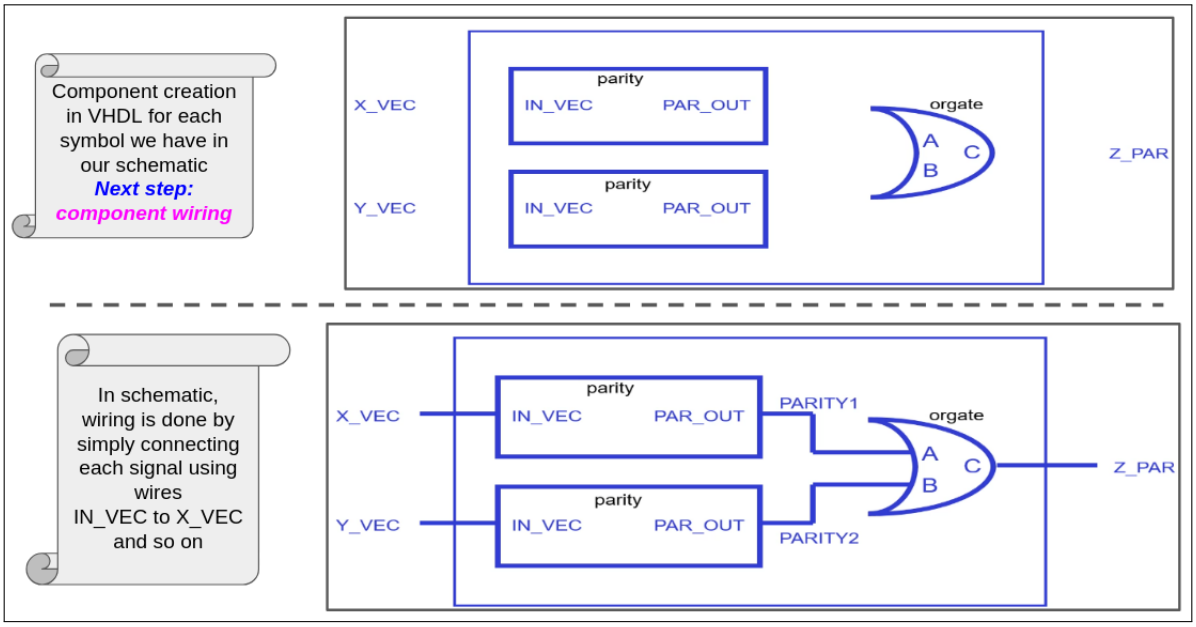


Figure 2.31: Symbols wiring

Creating top level desing

After finishing creating lower level of our system, that is creating component for each symbol, we start to implement the top level design, so we can have the VHDL program equivalent to the schematic of Figure 2.28.

1st step is to create the ***top level entity for the entire system***, as shown in Figure 2.32.

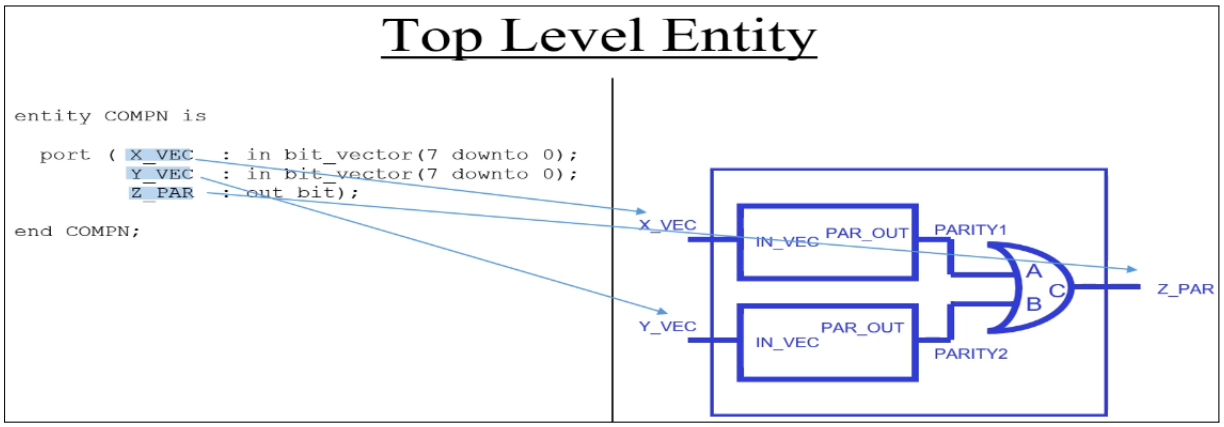


Figure 2.32: Top level entity

2nd step is the architectur part, which contained 2 steps.

1. ***Component Declaration***

   Because we want to use the component created in 2.6.1, `VHDL` need to declare them first. The way we do this is between `architecture` and `begin`. An example is shown in Figure 2.33 for the `OR` component.

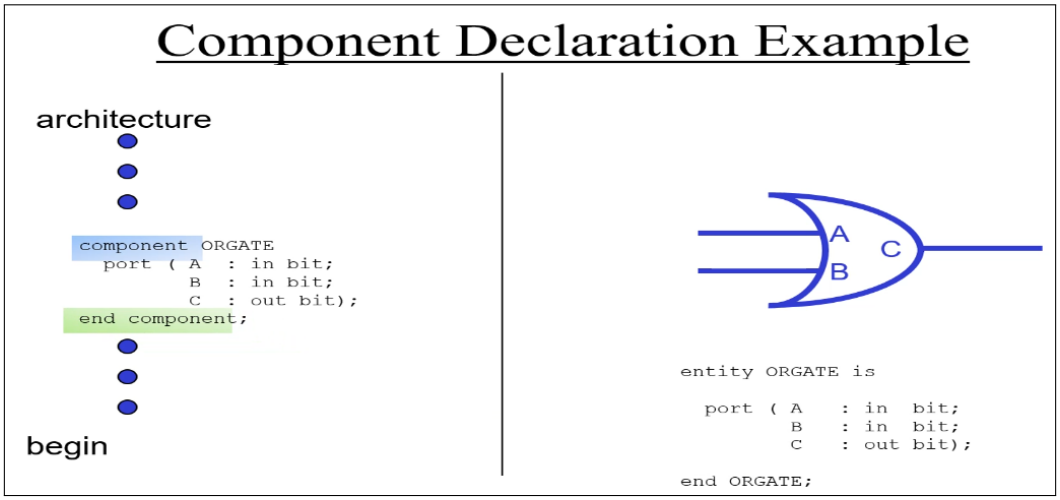


Figure 2.33: OR component declaration

- The component declaration is simply copy and past for the component entity with minor changes
  - `entity` is replaced `component`
  - `end ORGATE` is replaced by `end component`

Also in declaration, we declare any internal signal we need after so we can make connections as shown in Figure 2.34.

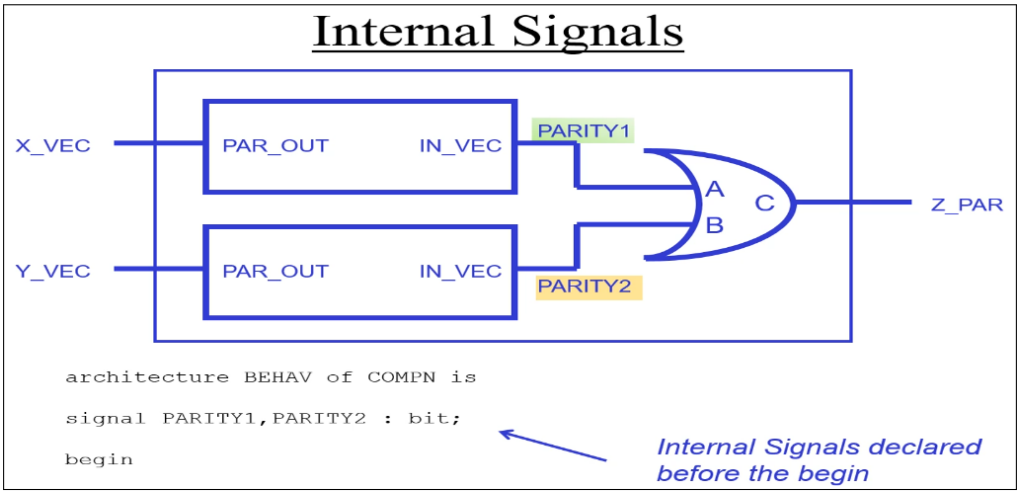


Figure 2.34: Internal signal declaration

2. ***Component Instantiation (Wiring part)***

   Once we declare the components between the `architecture` and `begin`, we need to instantiate the component, that is connect top level signals to component port. This is shown in Figure 2.35.

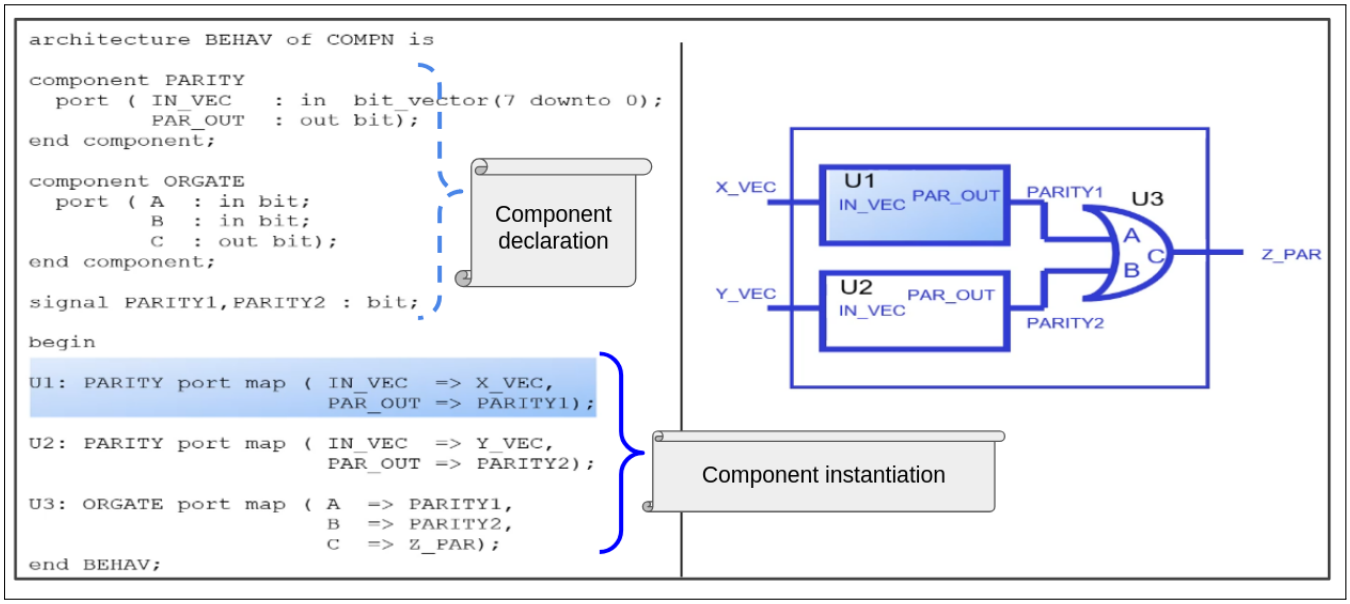


Figure 2.35: Components instantiation

- Here we have 3 component instantiated
- Each one of them has a ***label***: `U1`,`U2` and `U3`

When we instantiate a component, we map the pins of the component to the pins of the top level desing

- Component `U1`: its pins when creating this component are `IN-VEC` and `PAR-OUT`
- `IN-VEC` is mapped to `X-VEC`
- `PAR-OUT` is mapped to internal signal `PARITY-1`

The style of mapping we are using when instantiating a component is known as ***explicit mapping***, shown in Figure 2.36.

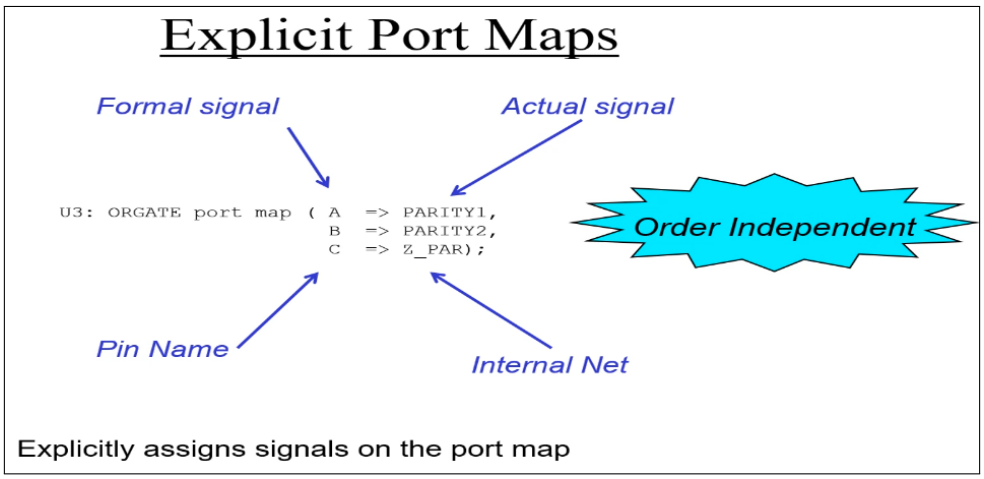


Figure 2.36: Explicit port mapping

- In formal `VHDL`, the pin name of the component created in a spearate file is known as *formal signal*
- The internal net belonging to the top level desing is called *actual signal*
- In this type of mapping, order is not important, since we are using formal signal and actual signal.

There is also what is called relative port mapping, where in the instantitation we list only the actual signals of the top level desing, and in this mapping order is important.

However in our code we use explicit since it gives more information (even if there is more typing).

2.7 Test bench

The concepts seen before in section 2.6 (component declaration and instantiation) are going to be used also in what is called *test bench*.

Test bench is a method to verify if our desing work properly or not. This is done by applying a set of *stimulus* to our input signals, and see if we obtain the correct output.

In Figure 2.37, we have an example how to create a testbench for the thermostat system we start it with.

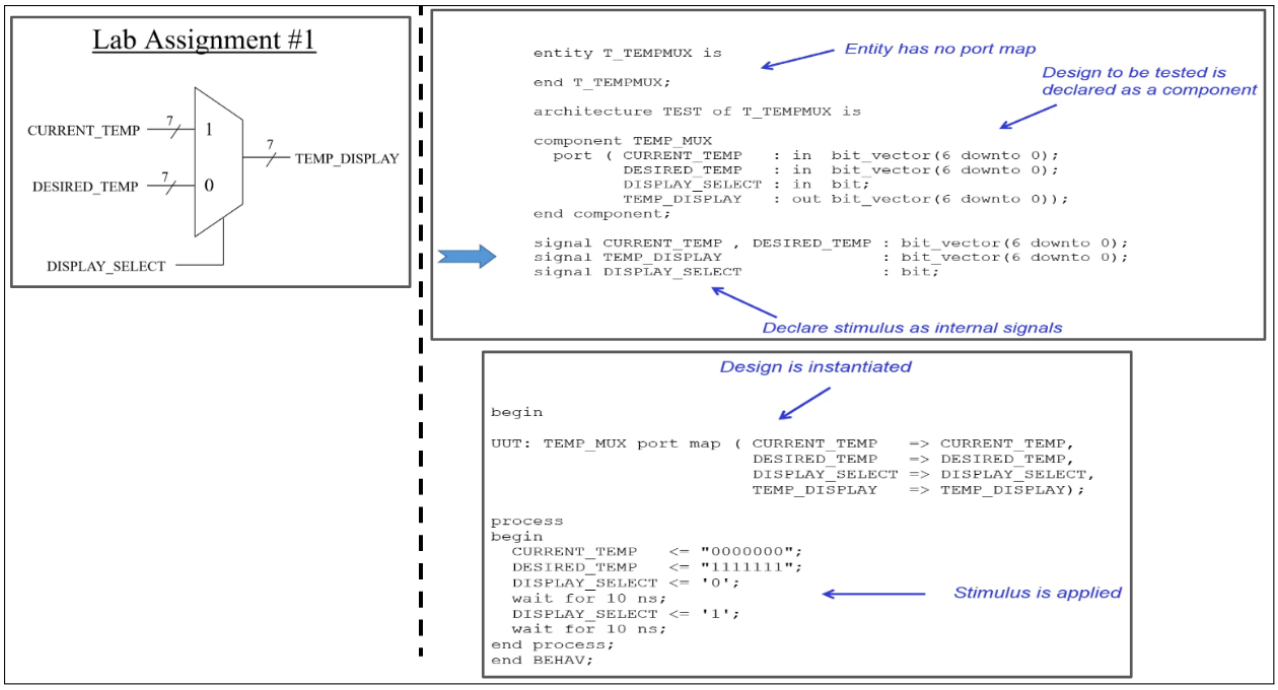


Figure 2.37: Test bench for the thermostat system

- A testbench is a separate file where our desing is being instantiated and used
- For testbench, the `entity` has no input or output port, since we are testing our design internally
- So as seen in section 2.6, we need to declare our desing first then instantiate
- The declaration is shown in the 2nd subfigure of Figure 2.37 (top right)
- Also in testbench: for each input signal we need to test it (apply some input value), we declare it as internal signal
  - That's why we have for example the 3 `signal` lines after component declaration
- After declaration, we instantiate the component and give it a label
  - it is labeled as `UUT`
- Note that in the instantiation of component in testbench is a special case
  - The pins of our component is not connected to some other top level actual signals as in Figure 2.36

  - The pins are connected to them self, since we are testing our component
- After isntantiation, we start a `process` with no sensitivity list
  - This makes the `process` start directly once the simulator starts
- Inside the `process`, we start apply some input values to some input signal, then use the statement `wait for ..`, and then apply some other value (change the input)
  - Here in the example we changed the value of `DISPLAY-SELECT`

2.8 Summary chap 2

2.8.1 Processes (2.2)

- 1st recall definition from chapter 1:
  - processe include the code which execute ***sequentially***
  - should contain a sensitivity list ↔ `process(A,B)` ↔ all inputs should be mentioned in the sensitivity list
- Type of interaction (2.2.1):
  - concurrent or sequential
    - recall from chapter 1, that when we work with concurrency, order of execution is not important since all of them will be executed in parallel using FPGA hardware capability
  - Depending on the sensistivity list (see Figure 2.1)
- Process rules (2.2.3)
  - sensitivity list should contains all the input
  - assignment happens at the end of the process
  - if multiple assignment to same signal, only the last one get executed

2.8.2 Data Types (2.3)

- Explore more data types such as integers and bool data
  - integers can support range of values (see example Figure 2.9)
- Litterale assignment (2.3.3)
  - depending on type of variables, we may use single quote (for bit), double quotation (bit vectors), or no quotation at all (boolean and integers)
- Vector comparison (2.3.4)
  - comparison is done from left to right, and bit by bit basis
  - We should pay attention to size of vectors (how many bits), and prevent vector mismatch
- Vector operation (aggregtaion and concatentation) (2.3.5)
  - concatenation uses `,`
  - aggregation usees `&`
  - The 2 operation lead to same result

- For aggregation: we have what is called explicit aggregation , that is assigning the indices of each vector to a certain value
- Recall from 1.7.1, that indices fo bit vectors are specfied using keyword `downto` or `upto`
  - example: 3 `downto` 0
  - index 3 is from left and 0 from right

2.8.3 Selection Control (2.4)

1. `if-else` (2.4.1)
   - the conditions comes with priority: from higest to lowest (see Figure 2.14)
   - It is preferred to implement a complete `if-else` to prevent latch syntehsis (see Figure 2.16)
2. `case` (2.4.2)
   - no priority as in `if-else`
   - we can't use bit vectors inside `when .. to` since bit vectors have no nuemrical meaning, they are just array of bits
   - MUX of `case` has lower gates, so if no priority is required, `case` is preferred
3. `for` loop (2.4.3)
   - rarely used in hardware
   - the synthesiser doesn't implement it as in normal programming language, instead it is ***unwrapped*** to its equivalent sequential statements
   - it can be used in circuit synthesis unlike other types of loop (`while` loop)
   - we can jump or break an iteration using `next when condition` keyword, or some equivalent `if not condition then`

2.8.4 Concurrent Statment (2.5)

1. `when ... else` (2.5.1)
   - comes with priority
   - the sequential equivalent is `if-else` inside a `process` (see Figure 2.24)
2. `with ... select` (2.5.2)
   - no priority required
   - sequential equivalent is `case` inside a `process` (see Figure 2.26)

2.8.5 VHDL Hierarchy (2.6)

- Idea is to make our work more modular when our work is starting to be bigger and larger

Generale steps taken from the parity system in 2.6.1:

1. Understand the system, and compose it to its lower details
   - That is how many component should we have to describe the entire system
   - In parity example, we used 3 components: 2 parity component and 1 `OR` gate component
2. Implement each component in a separate `VHDL` file
   - This will be normal `vhdl` coding, where each component will have its `entity` and `architecture`
3. After finishing all the component, we start by the *top level desing*. In this step, we have 2 steps:
   (a) Component Declaration
      - we delcare each component 1 time even if we use it multiple time
      - It is a simple copy and past of the `entity` of the component with replacement (`component OR end component`)
   (b) Component instantiation
      - We give our component a *label*
      - we do explicit port mapping: assigning formal signal (name of our signal in the component created) to the actual signal belonging to the top level desing

2.8.6 Test Bench (??)

- Test bench is a software method to verify if our desing work correctly
- Main idea: apply a set of input, and see if we obtain the correct output

General notes regarding test bench `VHDL` file:

- `entity` has no input or output port
- We need to declare and instantiate our component
- For all input signals we are going to test, we need to declare them as internal variables
- Component instantiation: port mapping is a special case
  - Pins are mapped to them self since we are goiing to test them